SCHOOL OF COMPUTER SCIENCE

# Exploration in Semi-Supervised ILM for Visual Meanings

Hyoyeon Lee

---

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of Bachelor of Science in the Faculty of Engineering **worth 40CP**.

---

Monday 12th May, 2025

# Abstract

Language is a distinctive feature of humans that enables the expression and understanding of thoughts. Understanding how language first emerged, how it is transmitted across generations, and what properties enable this transmission has been a long-standing interest in linguistics. One computational approach to address these questions is the Iterated Learning Model (ILM) [32] proposed by Kirby and Hurford. ILMs simulate the transmission and evolution of language across generations of learners; however, traditional ILMs require an obversion procedure when transitioning learners from pupils to tutors, which is computationally demanding. Bunyan, Bullock, and Houghton's Semi-Supervised ILM [8] addresses this limitation by implementing autoencoders for each agent, eliminating the need for obversion. Although this approach successfully demonstrates the emergence of compositional language structures with specific bottleneck configurations, it remains artificial and does not capture the variability characteristic of human cognition.

In this paper, we first replicate the Semi-Supervised ILM and verify its performance against the original results. Additionally, we develop a $\beta$-Variational Autoencoder ($\beta$-VAE) to process image meaning inputs. Building on this replication as a baseline, we propose an extension of the Semi-Supervised ILM that incorporates variational approaches to better reflect the human-like cognition, characterised by variability and uncertainty. Specifically, we replace the standard autoencoder with a VAE within the model. We evaluate the extended model's stability across generations under different training conditions (varying epochs, transmission bottleneck sizes, and $\beta$ values). Our results show that, while we do not observe the emergence of stable language under any specific hyperparameter conditions, the model demonstrates learning capabilities through latent feature representation and can accurately reconstruct inputs, occasionally making errors in one or two latent features. In general, we shed light on how incorporating cognitive variability through variational approaches affects language transmission and evolution, suggesting that while pure stability may not emerge, the model may partially capture more realistic aspects of human language learning.

# Dedication and Acknowledgements

I would like to express my deepest gratitude to my two supervisors, Dr. Seth Bullock and Dr. Conor Houghton, for their generous guidance, patience, and invaluable insights throughout this project. Despite having no obligation to meet with me regularly, they consistently devoted almost an hour every week to discussing my progress and offering advice. I am truly fortunate to have had the opportunity to work under their supervision; their support and encouragement have been fundamental in shaping my thinking and helping me navigate the challenges along the way.

I am also deeply grateful to my family for their unwavering support and belief in me, providing the motivation and reassurance I needed at every stage of this journey.

# Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Taught Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, this work is my own work. Work done in collaboration with, or with the assistance of others including AI methods, is indicated as such. I have identified all material in this dissertation which is not my own work through appropriate referencing and acknowledgement. Where I have quoted or otherwise incorporated material which is the work of others, I have included the source in the references. Any views expressed in the dissertation, other than referenced material, are those of the author.

Hyoyeon Lee, Monday 12$^{th}$ May, 2025

# AI Declaration

I declare that any and all AI usage within the project has been recorded and noted within Appendix A or within the main body of the text itself. This includes (but is not limited to) usage of text generation methods incl. LLMs, text summarisation methods, or image generation methods.

I understand that failing to divulge use of AI within my work counts as contract cheating and can result in a zero mark for the dissertation or even requiring me to withdraw from the University.

Hyoyeon Lee, Monday 12$^{th}$ May, 2025

# Contents

# List of Figures

# List of Tables

# Ethics Statement

This project did not require ethical review, as determined by my supervisor, Dr. Seth Bullock.

# Supporting Technologies

The following software and libraries were used for this study:

- `Python 3.11.4` and its standard libraries
- `PyTorch 2.5.1` to train neural networks
- `NumPy` for efficient numerical operations and array handling
- `random` to perform random sampling
- `matplotlib 3.9.2` to plot graphs and create visualisations
- `BluePebble` for running the project code

# Notation and Acronyms

| | | |
|---|---|---|
| ILM | : | Iterated Learning Model |
| BILM | : | Kirby and Hurford's Simple Iterated Learning Model |
| Semi-Supervised ILM | : | Bunyan, Bullock, and Houghton's Semi-Supervised Iterated Learning Model |
| VS-SILM | : | extended Semi-Supervised ILM with $\beta$-VAE developed in this project |
| AE | : | Autoencoder |
| VAE | : | Variational Autoencoder |
| CPH | : | Critical Period Hypothesis |

# Chapter 1

# Introduction

## 1.1 Motivation

Language is a unique feature of humans that enables one to express complex thoughts and understand others' intentions [22]. How the first language emerged, how it passes down through generations, and what properties of language makes it transmittable have long been subjects of interest in the fields of linguistics. Among many approaches used to find answers to these questions, computational modelling emerged as one of the prominent tools to simulate the process of language evolution.

One such model is Kirby's Iterated Learning Model (ILM) [30], which simulates iterative language transmission across generations that consists of tutor-pupil agent pairs for each generation. A tutor is an agent that has already acquired a language and is capable of mapping meanings to signals or back from signals to meanings based on its language knowledge. In each generation, the tutor teaches this language to a naïve pupil using the meaning-signal mapping it has developed through prior learning. After training, the pupil becomes a tutor in the next generation and passes on language to a new pupil, repeating the cycle across generations. Kirby and Hurford [32] demonstrated this methodology in their Basic Iterated Learning Model[1] (BILM), employing 8-bit binary strings as the language space, with a neural network as the decoder and a lookup table as the encoder. The BILM showed the emergence of a stable, expressive, and compositional language through iterative transmission, even without explicit modelling incentives towards compositional structure of language.

However, BILM shows several limitations which primarily lie in its dependence on a computationally expensive process known as *obversion*. Obversion is a process of training an agent's encoder to map meanings to signals by reversing the trained decoder, and exhaustively searching for optimal meaning-signal pairs. This process constrains the scalability and complexity of languages that can be simulated, making the language overly simplistic compared to actual human language. Additionally, the training process of the BILM consist solely of supervised training, which is unrealistic and not akin to how a child actually acquires language. Besides learning language based on a caregiver's direct instructions, a child also learns language via overhearing utterances that are related to current environment, and through self-reflecting what they would say to explain a concept.

To address these limitations, Bunyan et al. [8] introduced Semi-Supervised ILM, which employs a pair of neural networks configured as an autoencoder. Each network functions as an encoder and a decoder respectively, and the pupils are trained through a combination of supervised and unsupervised trainings. The Semi-Supervised ILM removes the need for obversion and improves the scalability of language by changing the structure of an encoder from a lookup table to a neural network that could be directly trained. Additionally, the model introduces an unsupervised training phase, which mimics the language learning of a child via observation and lifts the artificiality of the previous model's learning process. Notably, the Semi-Supervised ILM also achieved an emergence of stable, expressive, and compositional language, even with its newly introduced structure and training process.

Nonetheless, several challenges still remain. First, the meaning space is still based on binary strings, which is artificial and fails to capture the complexity of real-world semantic representations found in natural language. Second, standard autoencoders that are utilised in every agents perform a deterministic mapping between meanings and signals. This contrasts with the nature of human cognition, where indi-

[1] Although it is referred to as the Simple Iterated Learning Model in the original paper by Kirby and Hurford [32], we refer to it here as the Basic Iterated Learning Model to avoid confusion with the Semi-Supervised ILM introduced by Bunyan, Bullock, and Houghton [8], which will be discussed later in this project.

viduals may use different words to express the same perceptual input or describe the same environment, even when presented with identical stimuli [4].

These limitations are the central motivations of this project. We aim to explore the process language evolution, by extending Semi-Supervised ILM by integrating (1) more realistic meaning representations (e.g., visual inputs), and (2) a Variational Autoencoder (VAE) architecture into each agent. This proposed extension is significant for these reasons:

1. **Relevance to Human Language Acquisition**: Incorporating visual stimuli better reflects the nature of human perception, aligning the model more closely with how language is learned and used in natural settings.
2. **Probabilistic Representation of Meaning**: The use of a VAE enables the model to learn latent distributions rather than fixed encodings, capturing the inherent uncertainty and diversity of human linguistic expression.

Overall, this project aims to shed light on the variational state of Semi-Supervised ILM for language evolution by exploring a more cognitively and perceptually grounded framework for iterated learning.

## 1.2 Challenges of Adopting Image-Based Meanings

One of the major challenges in this project lies in extending ILMs from symbolic representations to high-dimensional visual inputs. Previous ILM frameworks [30, 32, 8] relied on $n$-bit binary strings to represent meanings, whereas the current project uses $64 \times 64$ binary images, which significantly increases the complexity and dimensionality.

This transition from fixed-length binary strings to high-dimensional images requires a substantial redesign of the agent architecture. The encoder and decoder networks must now process dense visual information, preserve relevant features in latent space, and ensure that learned representations support meaningful signal generation within this expanded representational framework.

Furthermore, the evaluation metrics for emergent language properties require fundamental redefinition. In the Semi-Supervised ILM, stability, expressivity, and compositionality were tractable and assessed through direct comparison of discrete binary meaning and signal due to equal dimensionality. However, such comparison no longer applies with image-based meanings due to the significant mismatch in dimensionality between meanings and signals. For instance, stability must now address language agreement between agents by measuring the level of image similarity.

The probabilistic nature of the VAE adds another layer of challenge to this project. Since VAEs encode meanings as distributions rather than fixed points, identical input may produce slightly varied outputs. While this is what we aim to better reflect the true nature of human cognition, this complicates how we evaluate the properties of language by requiring a plausible method that successfully distinguishes between inherent distributional variance and actual transmission instability. The strategies developed to address these challenges are outlined in more detail in Sections 3.3 and Section 3.4.

## 1.3 Objectives

The aim of this project is to investigate the ability of the original Semi-Supervised ILM when its meaning space is expanded from $n$-bit binary strings to high-dimensional $64 \times 64$ binary images. Furthermore, this project sheds light of bringing the model closer to simulating human cognition by replacing the standard autoencoder with a VAE. The following steps outline our approach to achieving this goal:

1. Replicate the Semi-Supervised ILM and validate its outcomes against the original results. Upon successful validation, this implementation will serve as a baseline for our project.
2. Train a baseline VAE on the dSprites dataset to confirm that it can capture structured latent representations and reliably reconstruct binary images in a controlled environment.
3. Integrate the trained VAE into the Semi-Supervised ILM and redesign the original evaluation metrics (e.g., stability, expressivity, compositionality) to accommodate visual and probabilistic representations.
4. Evaluate the emergent language of the new model. Based on the results, we either propose a viable extension of the Semi-Supervised ILM that preserves core linguistic behaviours observed in prior work, or identify specific limitations requiring further refinement to achieve our objectives.

# Chapter 2

# Background

## 2.1 Emergence of Language

Language is a uniquely human trait that sets us apart from other animal species. There are several key aspects that differentiate human language from animal communication. First, most forms of animal communication are more of a reflexive reaction to external stimuli [16]. In other words, animals lack the ability to combine and recombine signals to express new or complex meanings depending on the context. Additionally, the meanings conveyed in animal communication are either partly or entirely innate [37]. The majority of animals do not undergo a learning process to acquire signals commonly used within their own species, but are born with them. Human language, however, is different: it is highly syntactic, and acquired through learning and cultural transmission processes [32, 37]. Nonetheless, debate persists about whether certain aspects of language acquisition are genetically specified (see [12, 39] for nativist perspectives). These fundamental differences between language and communication have led to long-standing interest in understanding how the first language emerged, how it evolves through generations, and which properties enable it to be transmitted and acquired over time. Many approaches have been proposed to address these questions, among which computational modelling has become an increasingly prominent tool. Here, we focus on such computational approaches to shed light on the mechanisms underlying language evolution and transmission.

Kirby and Hurford [32] propose three systems, the intersection of which give rise to language:

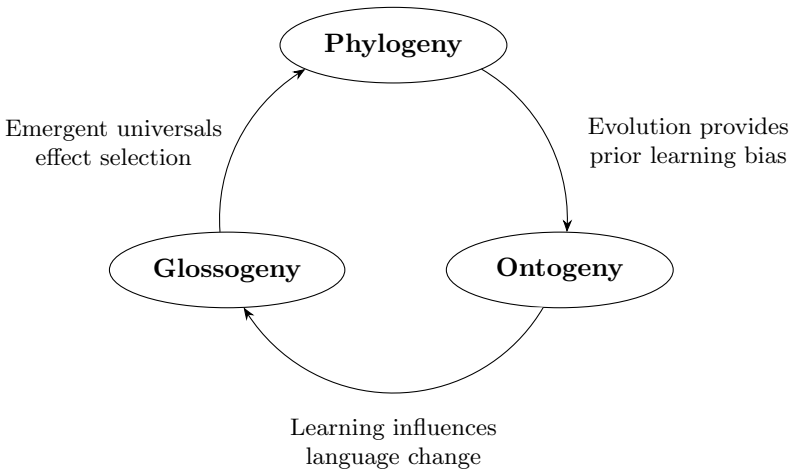



Figure 2.1: **The three adaptive systems that allow language to emerge.** *Phylogeny* refers to the biological evolution of the human capacity for language over long timescales. *Ontogeny* is the study of language in an individual over their lifetime. The key difference between phylogeny and ontogeny is that the former studies why humans can acquire language at all, while the latter is how each individual learns language with their biological equipment. *Glossogeny* is a term defined by Hurford [26], which refers to the cultural and historical development of individual languages over generations, as a result of accumulated experiences of individual language learners. The relationships between each system are stated, adapted from [32].

- **Learning**: During ontogeny, children continuously refine their linguistic knowledge through exposure to environmental input. This enables them to understand others and generate comprehensible utterances, especially during a developmental phase referred to as the *critical period*.
- **Cultural Evolution**: Language evolves as time passes, by the creation or extinction of a word, changes in its meanings, or adjustment in phonological/syntactic rules.
- **Biological evolution**: The human capacity for language has itself evolved, shaped by natural selection to optimise learning and processing mechanisms that enhance survival and reproduction.

The Critical Period Hypothesis (CPH) proposed by Lenneberg [35] posits that there exists a limited window during which the majority of language acquisition occurs, and learning a language after this period would be more effortful and less likely to achieve native-like proficiency. There has been a long-standing debate on whether or not the language acquisition is directly related to the biological development of human brain during childhood [5, 45, 13] and the answer still remains elusive, Hurford [27] suggests that there exists a relationship and models based under that assumption.

Exploring the interactive linguistic system illustrated in Figure 2.1 presents two primary challenges: an insufficient understanding of the interactions among its three constituent systems, and a lack of clarity in the methodological framework for exploration. To address such issues, computational modelling has emerged as a suitable approach for examining language transmission within a glossogenetic framework. One prominent method is the Iterated Learning Model [32], which simulates the cultural evolution of langauge through the observational learning and transmission of linguistic behaviour across successive generations.

## 2.2 Iterated Learning Model

The Iterated Learning Model (ILM) was first proposed by Kirby [30] to simulate how language changes during language acquisition across generations of artificial agents to discover under which conditions a compositional language is achieved. Upon completing a pupil's language training with its tutor, the pupil transitions into the role of tutor for a single new pupil agent in the next generation, and continues with a new language training process. This process reflects 'traditional transmission', as defined by Hockett [24], where language is acquired through learning rather than innate inheritance.

An ILM is specifically built under the idea that language acquisition in humans is constrained by a *critical period*, during which the majority of learning occurs – typically in the early stage before the end of puberty [27]. This assumption is reflected in the model through the introduction of a language bottleneck, which limits the amount of linguistic teaching that each pupil agents receives from its tutor. Hence, when a pupil becomes a tutor, it must often generalise utterances that go beyond the specific examples it encountered during training [32]. This process of generalisation plays a critical role in the behaviour of an ILM and is a key feature retained by many of their varients [41, 7].

According to Kirby and Hurford [32], a language can be formally defined as a map from a set of meanings $\mathcal{M}$ to a set of signals $\mathcal{S}$. This language representation is adapted from the distinction between E-language (meaning) and I-language (signal) proposed by Chomsky [11]. In this context, a *meaning* refers to a concept or an object as in internal representation, or as mentally encoded in one's brain, whereas a *signal* is an external language in the form of observable utterances to convey the desired meanings. With an ideal language, each agent in an ILM is capable of performing bidirectional mapping from any meaning $m \in \mathcal{M}$ to a unique signal $s \in \mathcal{S}$, and from any legal signal back to the meaning that generated it.

An ILM consists of four fundamental components: a meaning space $\mathcal{M}$, a signal space $\mathcal{S}$, a language-learning agent (typically only one per generation), and a language-teaching agent (typically only one per generation). In each iteration, a tutor is given a random list of meanings and must generate corresponding signals, thereby producing a set of meaning-signal pairs. These pairs are then used to train a naïve pupil agent, which initially has no language knowledge. After the teaching phase, the pupil transitions to become a tutor and must use their own language knowledge to generate a new set of signals from a new set of random meanings. The previous tutor is removed in order to maintain a constant population size, and the newly promoted tutor repeats this process with a fresh naïve pupil. Of the many variations of ILMs, this project follows Bunyan et al.'s [8] in using Kirby and Hurford's Simple ILM [32] as a basis for development. To avoid confusion with the semi-supervised extensions introduced by Bunyan et al., we hereafter refer to the original model as the Basic Iterated Learning Model throughout this work.

## 2.3 Basic Iterated Learning Model

### 2.3.1 Language and Model Architecture

In Kirby and Hurford's Basic Iterated Learning Model (BILM) [32], each agent is given a neural network decoder $d$ that maps signals to meanings, and a lookup table encoder $e$ that maps meanings to signals.

$$d : S \to M \tag{2.1}$$

$$e : M \to S \tag{2.2}$$

In a BILM, both meanings and signals are represented as 8-bit binary strings. Although the terminology originates from a later work [8], we adopt it here for clarity and ease of reference: a *fact* refers to an individual bit within a meaning, while a *word* denotes an individual bit within a signal. Each fact corresponds to a distinct feature of the concept being conveyed, and analogously, each signal corresponds to a specific syntactic element of the phrase. Having 8-bit binary strings gives 256 possible unique meanings and signals, and Figure 2.2 is one such example. The mappings between the signal and meaning varies greatly, where as there are only a few that are fully stable, expressive, and compositional. A stable language has consistent meaning-signal mappings; an expressive language can convey a wide range of meanings. A compositional language has grammatical structure, enabling complex meanings to be built from a simpler components. Detailed definitions of these metrics are provided in Section 2.5.

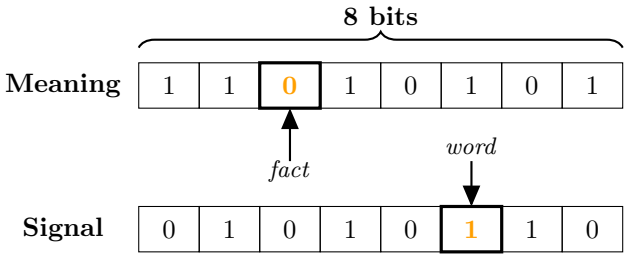



Figure 2.2: **An example pair of meaning and signal vectors**. Both the meanings and signals are 8-bit binary vectors, and each bit of them are referred to as fact and word respectively. For 8-bit binary strings, the range of possible meaning or signal is from 00000000 to 11111111.

A decoder $d$ is a fully connected three-layer feed-forward neural network with an $8 \times 8 \times 8$ architecture. In a feed-forward neural network, input information flows in a uniform direction from the input layer through a series of hidden layers to the output layer, without any cycles or feedback connections that return the output back to the earlier layers [18]. This directed, acyclic structure uses a sigmoid function (see Equation 2.3) as the activation function for nodes in all layers, which serves to constrain outputs to the range $[0, 1]$. This bounding is essential, as the final layer needs to avoid being saturated, and represent probabilities to reflect the network's degree of certainty regarding the presence of particular features within the target meaning. Additionally, the sigmoid function is appropriate for the model to perform backpropagation for stochastic gradient descent by being differentiable and having a computationally effective form of derivative.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{2.3}$$

The decoder produces an output vector of probabilities $\hat{d}(s) = (p_1, p_2, ..., p_n) \in [0, 1]^n$, where each $p_i$ represents the model's certainty that the $i$-th component of the meaning should be 1. To obtain an interpretable meaning, it is necessary to transform these continuous vectors into a discrete binary representation. To achieve this, a thresholding function $\delta$ is applied to determine whether each bit of the meaning should be 1 or 0 by:

$$\delta(p_i) = \begin{cases} 1 & \text{if } p_i > 0.5 \\ 0 & \text{otherwise} \end{cases} \tag{2.4}$$

which converts each probability $p_i$ into a corresponding bit of the meaning $m = (m_1, m_2, ..., m_n) \in \{0, 1\}^n$. Hence, the overall output of the decoder, is given by:

$$d = \delta(\hat{d}(s)) \tag{2.5}$$

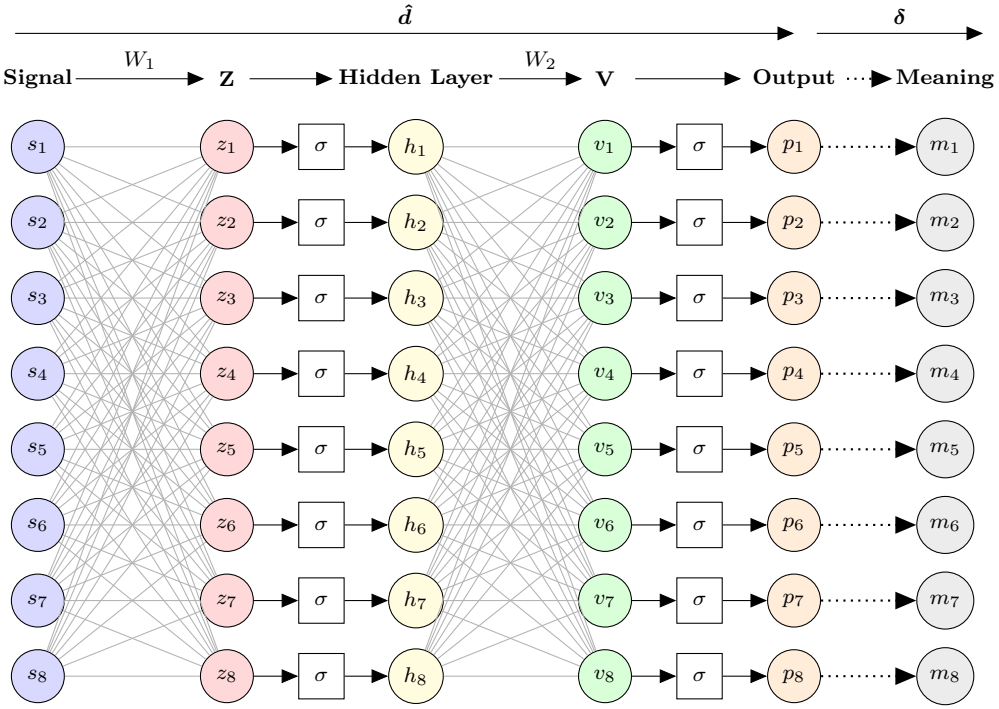



Figure 2.3: **Architecture of a neural network decoder $d$ in BILM**. A sigmoid function ($\sigma$) is used as the activation function to ensure that all outputs represent probabilities, i.e., values within the range $[0, 1]$. Every layer consists of eight nodes and is fully connected to every node in the subsequent layer. Layer $h$ and $m$ takes the weighted input from its former layer, with sigmoid function applied later. The hidden layer $h$ and the output layer $m$ compute their values by applying the sigmoid function to the corresponding weighted inputs. The layer $z$ and $v$ represent the pre-activation outputs of the hidden and output layers, which is the weighted sums before applying the activation function respectively. The weight matrices $W_1$ and $W_2$ map the nodes of one layer to the next along the forward pass of the network. This figure is based on Figure 1 from [41].

An encoder $e$ is represented as a $2^n \times 2^n$ lookup table that defines a predefined mapping from each of the $2^n$ meanings to a corresponding signal. An encoder is not directly trained as a parametric function akin to how the decoder is trained, but instead, is constructed through a process known as *obversion* proposed by Oliphant and Batali [38]. An obversion is a process of selecting the signal $s$ that the decoder most strongly associates with the given meaning $m$. The aim of this process is to find the signal that best corresponds to a given meaning by calculating the decoder's likelihood estimate $\hat{d}(m, s)$ for all possible meaning-signal pairs and identifying the signal with the highest confidence. In other words, it is evaluating for the intended meaning is $m$, how likely is that the decoder would produce that meaning from signal $s$. In a formal notation, for each meaning $m \in \{0, 1\}^n$, a desired signal $s' \in \{0, 1\}^n$ is assigned that maximises the following:

$$s' = \arg\max_s \hat{d}(m, s) \tag{2.6}$$

The likelihood $\hat{d}(m, s)$ is computed by feeding a signal $s$ into a decoder and treating the continuous network output as a measure of confidence in the presence of each meaning bits $m_i$.

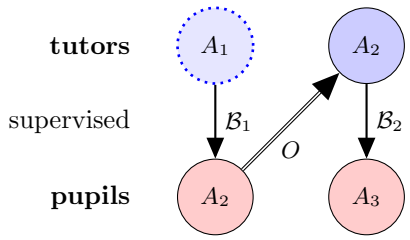
$$\hat{d}(m, s) = \prod_{i=1}^{n} p(m_i) \text{ where } p(m_i) = \begin{cases} p_i & \text{if } m_i = 1 \\ 1 - p_i & \text{if } m_i = 0 \end{cases} \tag{2.7}$$

For each bit $m_i \in \{0, 1\}$, if $m_i = 1$, the corresponding decoder output $p_i$ is expected to be high, and conversely, if $m_i = 0$, $p_i$ should be low. Assuming independence across meaning bits, these values are multiplied together to yield a joint probability. This, overall, measures the overall compatibility of signal $s$ with the desired meaning $m$. While conceptually straightforward, obversion introduces several drawbacks including its computational expense and limitations on the scalability of the language the

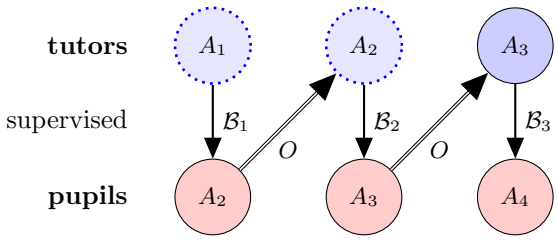
model could potentially use. This will be discussed in greater detail in the following section, along with the training process of a BILM.

### 2.3.2 Training Process

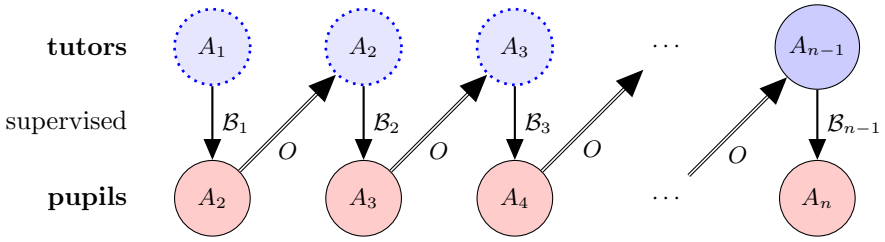
Training in a BILM starts with a tutor and a naïve pupil, the former equipped with a trained decoder and a fully established encoder, whilst latter equipped with a randomly initialised neural network decoder $d$ and an empty lookup table $e$. A tutor presents a pupil a subset $\mathcal{B}$ of signal-meaning pairs, sampled from the tutor's encoder with 100 randomised epochs of the dataset. The size of $\mathcal{B}$ is deliberately circumscribed, and smaller than the full meaning or signal space. This constraint is essential as it introduces a transmission bottleneck, as discussed earlier in Section 2.2, which compels the tutor to generalise beyond its prior knowledge. Given this set of mappings, the pupil trains its decoder on the subset $\mathcal{B}$ using stochastic gradient descent. Using sigmoid activation function is beneficial at this stage, as it is differentiable and has a well-behaved derivative while retaining nonlinearity, making it particularly well-suited for backpropagation. Once a pupil is done with training its decoder, it proceeds through an obversion procedure by reversing the decoder in order to derive an adequate encoder lookup table (see Table 2.1 for an example). Upon the completion of a pupil's training and construction of its encoder via obversion, it transitions into the role of tutor for the next generation. The original tutor is eliminated in order to strictly maintain the population size of two agents, and the cycle repeats over multiple generations to allow sufficient amount of time for structural changes in the language to be observed.




(a) **2-generation BILM with single obversion.**




(b) **3-generation BILM continuing from (a).**




(c) **Generalised training process of BILM.**

Figure 2.4: **Training process of the BILM.** Pupils are trained via supervised learning from the bottleneck dataset $\mathcal{B}_n$ passed down from its tutor, and after reconstructing the encoder by obversion ($O$) it transitions into a tutor for the next generation. The neural network is trained with a learning rate of $\eta = 0.1$ and no momentum term. Dotted nodes represent previous tutors which are removed once the model advances to the next generation. Solid arrows indicate supervised training of a pupil, while doubled arrows denotes obversion-based transitions from a decoder-only trained pupil to a fully trained tutor. This figure is adapted from Figure 2 from [8].

### 2.3.3 Evaluation and Limitations

In Kirby and Hurford's BILM [32], language is evaluated by two measures: stability, and expressivity. These metrics were used to assess the impact of the transmission bottleneck on the performance of emerged language. Interestingly, the language constantly converges on a compositional language that maps individual words to individual facts consistently despite there being many possible expressive languages, but only with an appropriate bottleneck size. As a result, the model successfully achieves a stable, expressive language as shown in Figure 2.5, albeit with two significant limitations [8]. First, the model is computationally expensive. The obversion procedure requires evaluating all possible signal-meaning

| $M \backslash S$ | 00 | 01 | 10 | 11 |
|---|---|---|---|---|
| **00** | 0.82 | 0.12 | 0.05 | 0.01 |
| **01** | 0.10 | 0.88 | 0.06 | 0.03 |
| **10** | 0.15 | 0.14 | 0.80 | 0.02 |
| **11** | 0.07 | 0.11 | 0.04 | 0.89 |

Table 2.1: **Example lookup table encoder $e$.** Each cell shows the probability $\hat{d}(m, s)$ estimated by the decoder of recovering meaning $m$ from signal $s$. The highlighted cells indicate the selected signal via obversion for each meaning.

pairs, resulting in a time complexity of $O(2^{2n})$. For an 8-bit binary string, this amounts to 65,536 evaluations; a modest increase to a 10-bits binary string would require over one million computations. This exponential growth severly restricts the language space to a modest scale and prevents the model from scaling to levels of complexity remotely comparable to real-world human language. Second, it is implausible that a child would mentally enumerate every possible signal and estimate the likelihood of it being used to express by every possible utterance. To address these limitations, Bunyan et al. [8] proposed a revised variant of the ILM that does not rely on an obversion procedure, yet still successfully produces a stable, expressive, and compositional language. This alternative framework will be discussed in the following section.

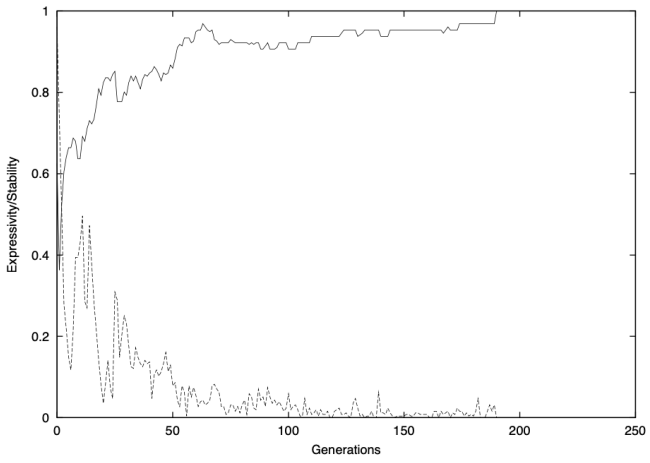



Figure 2.5: **The emergence of a stable and expressive language from BILM when the bottleneck is 2,000 random meanings**. The dotted line is stability, the size of the difference between learners and adults language after training. The solid line represents expressivity, which is the proportion of the meaning space covered by the language. This graph is directly taken from [32].

## 2.4 Semi-Supervised Iterated Learning Model

### 2.4.1 Language and Model Architecture

Bunyan, Bullock, and Houghton's Semi-Supervised ILM [8] was proposed in order to address the constraints of Kirby and Hurford's BILM. One of the key limitations of the traditional BILM was its lack of computational scalability, primarily due to its reliance on the obversion procedure. Semi-Supervised ILM addresses this by equipping each agents with an autoencoder, replacing the lookup table encoder with a trainable neural network. To understand this architectural change, we begin by examining the structure of the encoder.

As shown in Figure 2.6, the encoder maps a given meaning vector $m \in \{0, 1\}^n$ to a corresponding latent representation in the signal space. This process consists of two steps: first, the neural network outputs a probability vector $\hat{e}(m) = (q_1, q_2, ..., q_n) \in [0, 1]^n$, where each $q_i$ represent the model's confidence in whether the $i$-th word in the signal should be 1. Then, the thresholding function $\delta$ is applied to transform the continuous vector into a binary signal. The encoder is thus defined as:

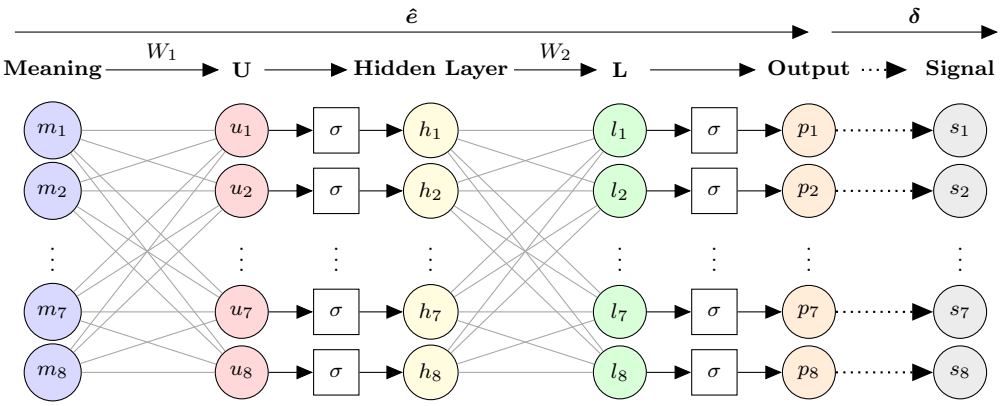



Figure 2.6: **Architecture of a neural network encoder $e$ in Semi-Supervised ILM**. The encoder shares the same architecture as the decoder described in Figure 2.3. It is fully connected $8 \times 8 \times 8$ feed-forward neural network with sigmoid activation and eight nodes per layer. The difference is that it maps from meanings to signals. Pre-activation function outputs (layers $U$ and $L$) and weight matrices ($W_1$, $W_2$) play the identical roles as in the decoder. $\delta$ is a thresholding function from Equation 2.4.

$$e(m) = \delta(\hat{e}(m)) \tag{2.8}$$

Building on the revised encoder structure, an autoencoder $a$ is a neural network designed to compress input data into a meaningful latent representation and then reconstruct the original input from this representation with minimal loss [3]. It consists of two sequentially connected neural networks (see Figure 2.7 for structure): an encoder $e$ that maps an input to the latent space, and a decoder $d$, which reconstructs the input from this given latent representation. The process first constructs an output probability vector $\hat{e}(m) = (q_1, q_2, ..., q_n) \in [0,1]^n$ and feed it back to the decoder. The decoder then processes this and produces another probability vector $\hat{d}(\hat{e}(m)) = (p_1, p_2, ..., p_n) \in [0,1]^n$ as reconstructed meaning, which is then applied $\delta$ to discretise it again. Hence, an autoencoder is defined as:

$$a = \delta \circ \hat{d} \circ \hat{e} \tag{2.9}$$

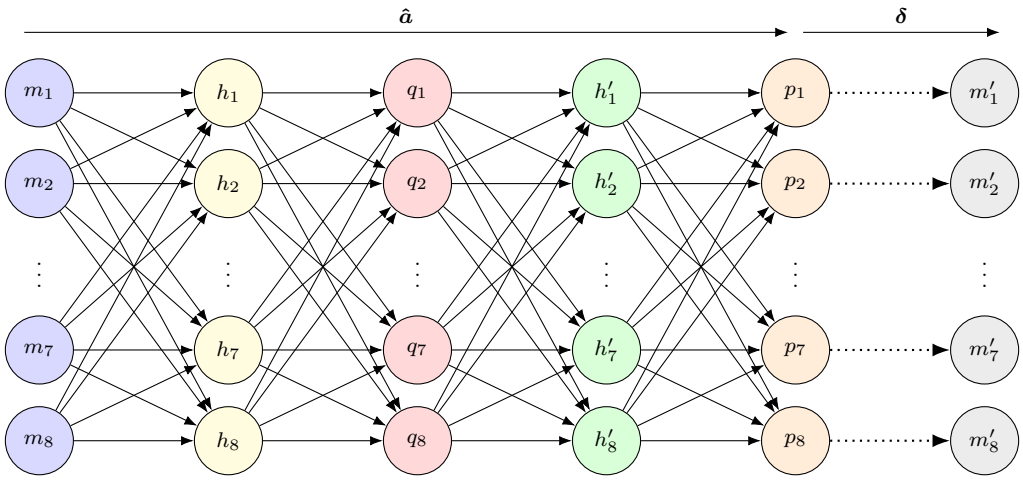



Figure 2.7: **Architecture of a autoencoder $a$ from a Semi-Supervised ILM**. The connections between nodes are represented by arrows, as it represent the application of function $\delta$. The autoencoder maps $\mathcal{M}$ to reconstructed meaning space $\mathcal{M}'$ by composing $\hat{e}$ and $\hat{d}$, passing through hidden layers $h_n$ and $h'_n$. The outputs of the encoder and decoder are probability vectors (e.g., $q_n$, $p_n$), where $p_n$ is discretised by function $\delta$ to transform it into a discrete vector. It is worth noticing that the signal layer is now one of the three hidden layers to concatenate $\hat{e}$ and $\hat{d}$ together. This figure is adapted from Figure 4 in [8].

### 2.4.2 Training Process

Much of language acquisition does not occur solely through *supervised* learning (direct instruction from adults), but also through processes resembling *unsupervised* learning. That is, children often acquire linguistic knowledge by overhearing utterances in context, such as utterances that are tied to the surrounding environment. The idea that language is learned not only through direct labelling or explicit explanations of a meaning but also via ambient exposure is supported by previous studies [28, 10, 2, 17, 1]. However, it is important to note that mere overhearing utterances is insufficient [39, 43]; there needs to be a mixture of both supervised and unsupervised input for a child to properly acquire a language.

In this context, a supervised input refers to direct instruction from an adult, often involving child-directed speech that provides explicit labels for objects or events. In contrast, unsupervised learning involves overheard speech or self-reflection, where the meaning must be inferred from context alone. This includes acquiring language by listening to utterances tied to the surrounding environment. Self-reflection, here, refers to observing events, imagining how they might be described, and evaluating those imagined description against one's observation [8]. The Semi-Supervised ILM reflects this hybrid learning scenario by training agents with a combination of supervised and unsupervised language exposure, thereby reflecting the diverse forms of linguistic ambience observed in a child's natural language acquisition.

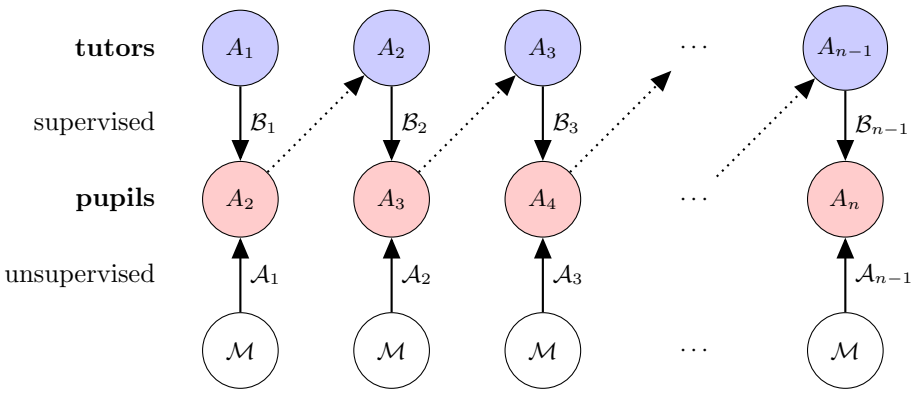


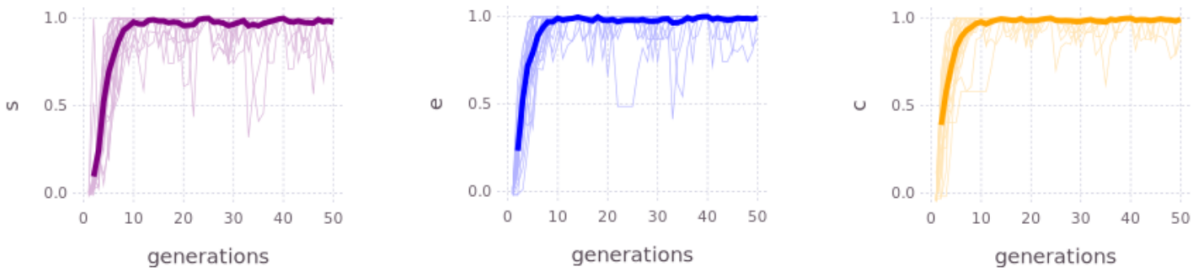

Figure 2.8: **Training process of Semi-Supervised ILM**. Semi-Supervised ILM also preserves a tutor-pupil architecture, but without strict population constraint. There exists two agents that are involved in each language acquisition. The tutor $\mathcal{A}_{i-1}$ produces a language transmission meaning-signal pairs $\mathcal{B}$ with its encoder $e_t$. $\mathcal{B}$ is presented to the pupil's encoder $e_p$ and the shuffled copy of $\mathcal{B}$ is presented to the pupil's decoder $e_p$ for supervised training. A set of meanings are randomly selected with replacement and is presented to $a$. This figure is adapted from Figure 3 in [8].

The model retains the tutor-pupil relationship of traditional ILMs. In each iteration of language acquisition, two agents are involved: a tutor $A_{i-1}$ and a pupil $A_i$. The tutor uses its encoder $e_t$ to generate a set of meaning-signal pairs $\mathcal{B}_i$, which reflects the bottleneck of a language transmission. This also means that the tutor, having itself received only limited language training from its own tutor, must generalise to previously unseen meaning-signal pairs. However, unlike earlier model that derived assumptions via decoder-based obversion, the tutor in the Semi-Supervised ILMs generates these pairs directly through $e_t$. The tutor presents $\mathcal{B}_i$ to the pupil in two ways for supervised training: one is given to the the pupil's encoder $e_p$, and a shuffled copy of $\mathcal{B}_i$ is handed out to the pupil's decoder $e_d$. Additionally, a separate set of meanings $\mathcal{A}_i$ is provided to the autoencoder $a$ over a specified number of epochs for unsupervised learning. Each complete cycle of these steps constitutes one epoch of the pupil's training.

Similar to BILM, the population is strictly restricted to two agents per every generation, with the tutor being eliminated once the pupil transitions into the tutor role.

### 2.4.3 Evaluation and Limitations

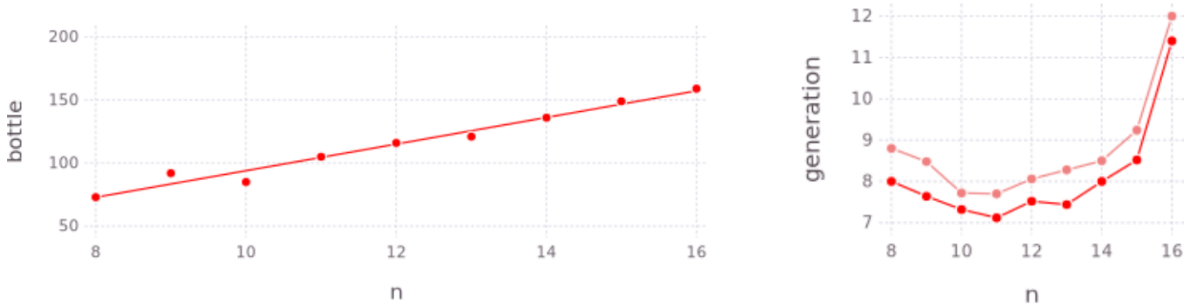
The Semi-Supervised ILM evaluates the language performance using identical metrics as the original BILM, with the addition of a compositionality measure. As shown in Figure 2.9, the model successfully achieves a XCS language (a language $x, c, s$ all exceeds $\lambda_g = 0.95$), despite the replacement of obversion with direct training under a hybrid learning framework. This demonstrates that a structured language can still emerge through a combination of supervised and unsupervised learning. The results from Figure 2.10 also shows that there exists a linear relationship between the dimensionality of meaning-signal space




(a) Stability (b) Expressivity (c) Compositionality

Figure 2.9: **The emergence of XCS language from Semi-Supervised ILM**. The performance was measured across 25 replicates of $n = 8$ Semi-Supervised Model where $|\mathcal{A}| = |\mathcal{B}| = 225$, and the bottleneck size is 75. The thick line shows the mean, and the thin lines indicates individual performance of each replicate. This figure is taken from Figure 5 in [8].

and effective bottleneck size and suggests that internal reflection on potential utterance is important in language learning and transmission.




(A) (B)

Figure 2.10: **The relationship between the bottleneck size and $\mathcal{M}$ size**. The model is run until $x, c, s$ reaches $\lambda = 0.95$ with 25 independent replicates. **(A)** demonstrates a linear relationship between $\mathcal{M}$ size ($2^n$) and the number of sufficient bottleneck size to reach a desired $x, c, s$ level, with a slope 10.5 and intercept -11.6. **(B)** illustrates the average number of generations used to reach $\lambda_g = 0.95$. The pink line illustrates the average number of generations required when using a value one away from the optimal bottleneck size. The red line shows the same measurement but with the optimal bottleneck size.

However, a key limitation of the model lies in its use of $n$-bit binary vectors to represent meanings and its use of standard autoencoders. While effective in controlled situations, these vectors are overly simplistic and are far from capturing the natural complexity of human language [34]. Also, the deterministic nature of standard autoencoders enforces a rigid, one-to-one mapping between meanings and signals. In contrast, human cognition is inherently flexible, with the ability to generalise from prior experiences to new, unseen examples. In other words, children must be able to categorise objects into "kinds" (nouns such as 'dog', 'ball', 'tree') in order to acquire linguistic conventions, which requires flexible generalisation and variability, even for unseen information [46, 21]. According to Barsalou [4], individuals represent concepts through partial reactivations of sensory experiences shaped by prior encounters. For example, if a child who has already seen several types of dogs before is shown an image of an unfamiliar dog breed and asked, "What is this?", they will likely recognise it as a dog, even without having seen that breed before. Additionally, if someone asks two children to think of a dog, the mental image they think of is unlikely to be identical in a literal, image-level sense, while sharing key conceptual features, e.g., four-legged, barks, has a tail. Such findings underscore the need for models that could capture uncertainty and representational variability [40], and a structure capable of incorporating prior knowledge or experiences into present decision making. To address this limitation, we introduce Variational Autoencoder (VAE), which will be discussed in Section 2.6.[1]

[1] Although Barsalou (1999) does not formalise perceptual simulations in statistical terms, the emphasis on variability and flexible reactivation aligns conceptually with probabilistic generative models like VAEs, which account for uncertainty and generalisation across perceptual space.

## 2.5 What Is A Good Language?

To know whether a language is structured and comparable to human language, metrics are needed for evaluation. Bunyan et al. [8] adopt three fundamental properties that can be used to evaluate and quantify a performance of a language : stability ($s$), expressivity ($x$), and compositionality ($c$). Each measure ranges from 0 to 1, and a language where all three exceed a threshold of $\lambda = 0.95$ is classified as a XCS language. What each property represents and how to measure it are described below.

Stability quantifies to which degree two agents agree in their use of language. That is, measuring the likelihood that when one agent encodes a meaning into a signal using its own encoder, another agent can successfully decode that signal back into the original meaning using its own decoder.

$$s = \frac{|\{m \mid m = d(e(m))\}|}{|M|} \tag{2.10}$$

This property is evaluated upon the formation of a tutor and a newly trained pupil pair, specifically at the conclusion of the training process for a previously naïve pupil, where $d$ is a decoder of one agent, and $e$ is an encoder for another. The value of $s$ being closer to 1 would indicate more stable language, whereas it leaning to 0 would mean that the language does not agree well between two agents, potentially failing to be a good language.

Expressivity measures how many unique signals the agents use to express each of the possible different meanings. In other words, it evaluates the extent to which an agent utilises the available signal space to represent all the meanings in $\mathcal{M}$. The opposite of expressivity is ambiguity when one signal is used to represent more than one different meaning. We can calculate $e$ by passing the entire $2^n$ size of the meaning set and counting the number of unique signals generated by the agent's encoder, divided by the size of the signal set. A value of $e$ being close to 1 indicates that the agent is capable of assigning a unique signal to each meaning.

$$x = \frac{|\{e(m) : m \in M)\}|}{|S|} \tag{2.11}$$

Compositionality refers to the extent to which complex expressions are systematically constructed from simpler, meaningful components. In the context of language, a highly compositional system consistently maps meanings to signals such that each individual fact is represented by a distinct word. Hence, words are not used ambiguously or unpredictably across different facts. The opposite of compositionality is arbitrariness–where similar meanings are mapped to arbitrarily different signals and dissimilar meanings may be mapped to signals that share several components. The evaluation framework adopted in the original study draws on Frege's principle of compositionality [15], which posits that the meaning of a complex expression is determined by the meanings of its parts and the rules used to combine them. An example of a fully compositional language is provided in Table 2.2.

| **Meaning** | **Signal** |
|---|---|
| (0,0) | (0,0) |
| (0,1) | (0,1) |
| (1,0) | (1,0) |
| (1,1) | (1,1) |

Table 2.2: **Signal to Meaning Mapping**. The column on the left represents the 2-bits signal space and the column on the right represents the 2-bits meanings associated with each signal. This is the simplest language since it is a one-to-one mapping where the bits of the signal are the same as in the meaning. It is fully compositional since, for each column, a 0 in the signal represents a 0 in the meaning.

As established earlier, meaning is represented as a vector $m = (m_1, m_3, ..., m_i, ..., m_n)$ where each $m_i$ is a fact. A signal $s$ consists of words $s = (s_1, s_2, ..., s_j, ..., s_n)$. The objective is to assess whether each fact $m_i$ maps uniquely to a specific word $s_j$. In order to do this, an entropy is computed for each fact-word pair. Entropy quantifies the uncertainty of mapping: the higher the entropy, the less deterministic association. Given probability $p(s_j = 1|m_i = 1)$, the entropy $h_{ij}$ is defined as:

$$h_{ij} = -p \log_2 p - (1-p) \log_2 (1-p) \tag{2.12}$$

A value of $h_{ij}$ close to 1 indicates that the fact $m_i$ is distributed roughly evenly across multiple signal components, reflecting a random and inconsistent mapping. Conversely, $h_{ij} = 0$ implies that each fact $m_i$ consistently map to a single, specific word $s_j$, corresponding to a fully deterministic language.

For each fact, the word that yields the lowest $h_{ij}$ is selected, thereby identifying the most reliable mapping across all possible words. Hence, every fact is paired with the word that provides the most deterministic word in the language.

$$H = \{h_{ij} \mid h_{ij} = \min_k(h_{ik}) \text{ for } i = 1, \ldots, n\} \tag{2.13}$$

In cases where multiple facts are mapped to the same word (i.e., collisions), the word is assigned to the fact that has lower entropy, while the conflicting entries are penalised by setting their entropy values to 1.0 to reflect the ambiguity in unique mapping. The final entropy measure is thus defined as:

$$h'_i = \begin{cases} \min_j h_{ij} \in H & \text{if } |\{h_{ij} \in H\}| \neq 0 \\ 1 & \text{otherwise} \end{cases} \tag{2.14}$$

Finally, the compositionality value $c$ is calculated by:

$$c = 1 - \frac{1}{n} \sum_i h'_i \tag{2.15}$$

## 2.6 Variational Autoencoder

While standard autoencoders resolve computational expense and move us closer to human-like learning, they retain limitations in their artificiality that posits from deterministic mapping between meanings and signals, as observed in Section 2.4.3. The Variational Autoencoder (VAE) [29] address these limitations by casting the problem as 'generative modelling': they aim to capture the underlying data distribution of high-dimensional signals (e.g., images) [14]. The objective is to learn a distribution $P(X)$ over data $X$ that not only assigns plausible likelihoods to observed inputs but also generates novel samples that resemble the inputs observed during training. In practice, we often aim to generate diverse, realistic samples that are akin to inputs but not identical. While some approaches aim to estimate $P(X)$ explicitly or numerically, practical applications often require more than mere likelihood estimation: we seek to generate diverse, realistic data samples that are similar to those seen during training, but not identical. This stochastic nature of the VAE's latent space introduces both variability and uncertainty, effectively addressing the earlier limitations of limited expressiveness and rigid determinism. How VAEs achieve this will be explored in more detail in the sections that follow, based on the foundational descriptions in [29, 14].

### 2.6.1 Latent Variables

To build a clear understanding of Variational Autoencoders (VAEs), it is helpful to first understand the concept of a 'latent variable'. Latent variables are unobserved variables that capture underlying structure in the data, influencing observable outcomes without being directly measured. For instance, when generating an image of a handwritten digit, the model might internally decide which digit it indents to create (such as 3 or 7) before determining the specific pixel-level details. Although such internal decisions are not explicitly encoded in the image, they heavily influences what the image looks like. Such hidden factors are referred to as latent variables, often denoted as $z$ and the latent space as $\mathcal{Z}$.

A key challenge when designing models with latent space is deciding how to define the latent variables $z$ in a way that captures meaningful, structured information about the data. For example, in the context of image generation, latent variables might represent properties such as the digit identity (i.e., the number generated), the angle at which it is written, and stroke thickness. These properties are often entangled in complex ways–for example, a more slanted digit might result from being written quickly, which in turn may lead thinner strokes. Attempting to assign a specific meaning to each dimension of $z$, or to manually encode the relationships between them, quickly becomes impractical as data complexity increases. VAEs resolve this issue by imposing an assumption that the latent variables are drawn from a standard multivariate Gaussian distribution $\mathcal{N}(0, 1)$, where each dimension is independent and identically distributed. This is grounded in a probability theory that complex distributions in high-dimensional spaces can be obtained by applying sufficiently expressive transformations to samples from simple distributions. In practice, VAEs leverage this principle by using a neural network decoder $f(z; \theta)$ (which

is $\hat{d}$ accordingly to our previous notations), which is parameterised by weights $\theta$, to learn how to map latent samples $z$ into complex and structured data. During training, the model learns how to associate different regions of the latent space with different semantic characteristics of the data without needing any explicit supervision or manual labels. As a result, the decoder learns to interpret the latent space in a meaningful way through the pressure of reconstructing inputs accurately and remaining consistent with the prior. The first few layers of the decoder may implicitly map the standard normal variables into useful representations, while subsequent layers render the final output. In this way, VAEs avoid the need to engineer the latent space and instead let the model discover structure in the data, guided by the objective of maximising the likelihood of the training examples.

Within this framework, we assume that for every data point $x$ in the dataset $X$, there exists at least one latent representation $z$ such that passing $z$ through a decoding function yields a close approximation to $x$. To formalise this, we define a prior distribution over the latent space $P(z)$, typically a Gaussian $\mathcal{N}(0, 1)$, that enables principled and efficient sampling of latent variables during both training and generation. The decoder function $f(z; \theta)$ is deterministic: given a fixed $z$ and parameters $\theta$, it always outputs the same reconstruction. However, since the input to the decoder is sampled from a distribution, the overall process of data generation is stochastic. Recall that the model's goal is to generate outputs that resemble the observed samples by maximising the likelihood of the training data under this generative process. This likelihood is formally defined as the marginal probability of the data, obtained by integrating over all possible latent variables:

$$P(X) = \int P(X|z; \theta)P(z)dz \tag{2.16}$$

where $P(X|z; \theta)$ is the likelihood of observing $X$ given a latent variable $z$:

$$P(X|z; \theta) = \mathcal{N}(X|f(z; \theta), \sigma^2 I) \tag{2.17}$$

The likelihood $P(X|z; \theta)$ is typically modelled as a Gaussian distribution centred at $f(z; \theta)$ with covariance $\sigma^2 I$ (scaled identity matrix $I$). This enables the model to generate diverse outputs by sampling from the latent space and ensures a continuous, structured mapping from latent variables to $\mathcal{X}$. However, using Equation 2.16 to compute the marginal likelihood $P(X)$ becomes incredibly difficult in high-dimensional spaces, as it involves integrating over all possible values of $z$. VAE address this challenge via approximate inference using the 'Evidence Lower Bound (ELBO)', which is described in detail in the following section.

### 2.6.2 Evidence Lower Bound

Recall that the overall objective of a VAE is to maximise $P(X)$ of the data, as shown in Equation 2.16. We can optimise this directly using stochastic gradient ascent, under the condition there exists a computational equation for $P(X)$ which is also differentiable. Estimating $P(X)$ in theory, is actually conceptually straightforward: one could approximate it by sampling a large number of latent variables from the prior distribution $P(z)$ and compute $P(X) \approx \frac{1}{n} \sum_i P(X|z_i)$. However, this approach becomes inefficient and impractical in high-dimensional latent space, as the probability mass of high-dimensional Gaussian prior tends to concentrate in narrow regions. This makes it extremely unlikely that random samples from $P(z)$ will land in regions of latent space that actually lead to meaningful reconstructions of the input $X$. As a result, this requires a great number of samples just to get a decent estimate of $P(X)$.

To address this issue, VAEs adopt variational inference. Rather than trying to compute the intractable true posterior $P(z|X)$, the model introduces a more manageable approximation: posterior $Q(z|X)$, which is usually parameterised by a neural network. Here, $X$ is fixed, and the expectation is taken over samples $z \sim Q(z|X)$. It is important to know that $Q(z|X)$ does not have to exactly match the true posterior $P(z|X)$; it just needs to be close enough to allow effective learning. The divergence between $Q(z|X)$ and $P(z|X)$ is measured using the Kullback-Leibler (KL) divergence:

$$\mathcal{KL}[Q(z)||P(z|X)] = \mathbb{E}_{z\sim Q}[\log Q(z) - \log P(z|X)] \tag{2.18}$$

Using Bayes' theorem, we can now rewrite $\log P(z|X)$:

$$\log P(z|X) = \log P(X|z) + \log P(z) - \log P(X) \tag{2.19}$$

Substituting this into the KL divergence, we obtain the following:

$$\log P(X) = \mathbb{E}_{z\sim Q}[\log P(X|z)] - \mathcal{KL}[Q(z)||P(z)] + \mathcal{KL}[Q(z)||\log P(z|X)] \tag{2.20}$$

Since the KL divergence is always non-negative, this leads directly to a lower bound on the marginal likelihood:

$$\log P(X) \geq \mathbb{E}_{z\sim Q}[\log P(X|z)] - \mathcal{KL}[Q(z|X)||P(z)] \tag{2.21}$$

This inequality is referred to as the Evidence Lower Bound (ELBO), which consist of two parts:

- $\mathbb{E}_{z\sim Q(z|X)}[\log P(X|z)]$: encourages the model to accurately reconstruct the input from the latent representation.
- $\mathcal{KL}[Q(z|X)||P(z)]$: regularises the latent space by encouraging the approximate posterior to remain close to the prior $P(z)$, usually set as $\mathcal{N}(0, I)$ when $I$ is an identity matrix.

Together, these terms form the final objective that is optimised during training, followed as:

$$ELBO(X) = \mathbb{E}_{z\sim Q(z|X)}[\log P(X|z)] - \mathcal{KL}[Q(z|X)||P(z)] \tag{2.22}$$

This formulation is central to the VAE framework, as it provides a principled mechanism for balancing data reconstruction with latent space regularisation. By maximising the ELBO, the model simultaneously learns to approximate the true data distribution while encouraging the latent representations to exhibit desirable properties such as smoothness and continuity. In practice, both the encoder $Q(z|X)$ and the decoder $P(X|z)$ are implemented using neural networks and are updated jointly to maximise the ELBO across the training dataset. Despite the ELBO offering a tractable objective for training, it introduces a practical challenge during optimisation via stochastic gradient descent. Specifically, the ELBO involves sampling from $Q(z|X)$, and this sampling operation is inherently non-differentiable. As a result, it is not straightforward to propagate gradients through the sampling process using standard backpropagation. To address this, VAEs employ a technique known as the reparameterization trick to reformulate the sampling process in a differentiable manner, which will be discussed in detail in the following section.

### 2.6.3 Reparameterization Trick

As previously discussed, one challenge in optimising the ELBO arises from the need to compute gradients through a sampling operation from the approximate posterior $Q(z|X)$. Since the VAE's encoder outputs a distribution rather than a single point estimate, this stochastic component hinders straightforward gradient-based optimisation. That is, because the ELBO includes an expectation over latent variables $z \sim Q(z|X)$, the non-differentiable sampling step complicates the backpropagation process.

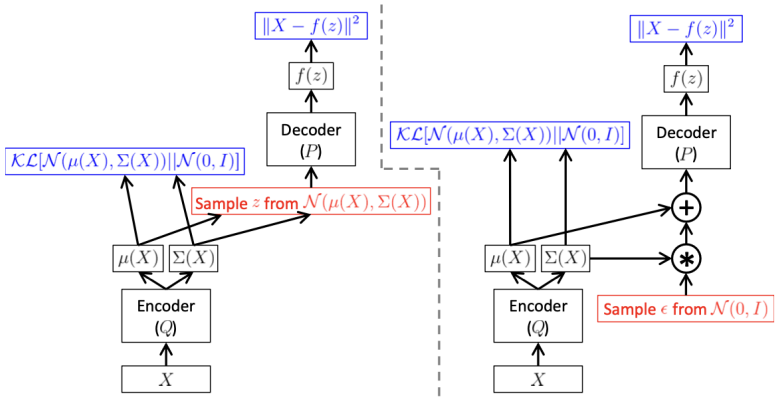



Figure 2.11: **Implementations of VAEs with and without reparameterization trick, as a feedforward neural network and $P(X|z)$ being Gaussian**. **Left** is without reparameterization trick, where $z$ is sampled directly from the learned distribution $(\mathcal{N}(\mu(X), \sigma(X))$, making it difficult to propagate gradients through the sampling step. **Right** is with the reparameterization trick, where $z$ is computed using Equation 2.24 and $\epsilon \sim \mathcal{N}(0, I)$. Red box indicates sampling operations that are non-differentiable, while blue box shows loss layers. The feedforward behaviour of these networks is identical, albeit backpropagation can only be applied to the right network. This figure is directly taken from [14].

The reparameterization trick addresses this issue by decoupling the source of randomness from the distribution's parameters. Instead of sampling $z \sim Q(z|X)$ directly, we express $z$ as a deterministic function of a noise variable that is independent of the learnable parameters. Formally, for a continuous latent variable $z$, we reparameterise the sampling process as $z = g(\epsilon, X)$, where $e \sim P(\epsilon)$ is drawn from a fixed, parameter-free distribution, and $g$ is a differentiable transformation. Commonly, when $Q(z|X)$ is modelled as a multivariate Gaussian distribution with diagonal covariance such as:

$$Q(z|X) = \mathcal{N}(z|\mu(X), \sigma^2(X) \cdot I) \tag{2.23}$$

we can reparameterise $z$ as:

$$z = g(\epsilon, X) = \mu(X) + \sigma(X) \odot \epsilon \tag{2.24}$$

where $\epsilon \sim \mathcal{N}(0, I)$. Here, $\mu$ and $\sigma(X)$ are outputs of the encoder network, representing the mean vector and the standard deviation vector of the latent distribution respectively, while $\odot$ denotes element-wise multiplication (see Figure 2.11). This formulation makes the dependence on $X$ explicit while isolating the stochasticity in $\epsilon$, which is independent of the model parameters. As a result, we can compute gradients with respect to $\mu(X)$ and $\sigma(X)$ via standard backpropagation through the deterministic function denoted as in Equation 2.24. Intuitively, this reparameterization can be seen as moving the randomness outside the network, thereby allowing gradient flow to be preserved during training. It is important to note that this trick relies on the assumption that the latent distribution is reparameterisable, which is a condition satisfied by many common distributions, such as the Gaussian.

# Chapter 3

# Method

## 3.1 Replicating the Semi-Supervised ILM

The project starts by replicating the Semi-Supervised ILM as a baseline and confirming its performance as described in [8]. Due to the flexibility and convenience of `PyTorch` in constructing neural networks, the project is developed using `Python` (whereas the original paper reports on an implementation constructed in Julia).

### 3.1.1 Language Generation

As established in the original paper, the Semi-Supervised ILM uses 8-bit binary strings to represent both meanings and signals. The supervised dataset $\mathcal{B}$ is created by selecting a fixed set of 75 random meanings with replacement and generating corresponding signals using the tutor's encoder, thereby forming meaning-signal pairs for training pupils. This dataset is used to train the encoder, whilst a separate shuffled copy, $\mathcal{B}'$, is used to train the decoder. Concurrently, the unsupervised dataset $\mathcal{A}$ is generated by randomly sampling 75 random meanings from the meaning space $\mathcal{M}$ with replacement.

### 3.1.2 Agent Architecture and Training Process

Each agent in the simulation is defined as a `Python` class and consists of five key components, as illustrated in Table 3.1. Conceptually, an agent includes a meaning-to-signal (`m2s`) network, a signal-to-meaning (`s2m`) network, and internal parameters such as `bitN`, which specifies the length of the meaning and signal strings, and `num`, the agent's index in the population. All neural networks are implemented using `PyTorch`, which provides a flexible framework for defining network architectures and simplifies model training through automatic differentiation. Both `m2s` and `s2m` networks consist of three linear layers with sigmoid activation functions. When agents are first initialised, network weights are randomly assigned, with no pre-training or architectural constraints.

| **Agent** |
|---|
| `+ bitN: int` |
| `+ num: int` |
| `+ m2s: NeuralNetwork` |
| `+ s2m: NeuralNetwork` |
| `+ m2m: nn.Sequential(m2s, s2m)` |

Table 3.1: **The UML class diagram of `Agent` class**. `bitN` and `num` each corresponds to the length of binary string inputs and the agent index. Both forward and backpropagation steps are not encapsulated into separate class methods, but are implemented directly within the training logic.

As shown in Algorithm 3.1, the training process is as follows. The first agent is considered mature and acts as the initial tutor, despite not having a tutor of its own. Before training begins, a meaning space $\mathcal{M}$ is constructed, where meanings are created by converting all integers from zero to $2^n - 1$ into binary bits as in tensors. Based on this, the tutor generates the following datasets for training each pupil:

- $\mathcal{A}$: an unsupervised dataset consisting of 75 randomly sampled meanings.
- $\mathcal{B}_1$: a bottleneck supervised dataset created by sampling 75 meaning-signal pairs with replacement from full supervised dataset $\mathcal{T}$ .
- $\mathcal{B}_2$: a shuffled copy of $\mathcal{B}_1$.

At each training iteration, these datasets are used as follows. For each index $i$ in the batch, the $i$-th item from $\mathcal{B}_1$ is used to train the decoder $\hat{d}$, and the $i$-th item from $\mathcal{B}_2$ is used to train the encoder $\hat{e}$. Additionally, a random subset of $r = 20$ meanings is selected from $\mathcal{A}$ and used for unsupervised training of the autoencoder $\hat{a}$. Once training is complete, the pupil is evaluated using XCS metrics, which quantify the stability, expressivity, and compositionality of the learned language. The pupil then becomes the next tutor for next generation, and a naïve pupil is generated. This iterated learning process continues for 50 generations.

**Input:** Agent, Tutor, *A_size*, *B_size*, Meanings, Epochs
Initialise optimisers for encoder, decoder, and autoencoder
Set loss function to MSE
Generate supervised dataset $\mathcal{T}$ and unsupervised dataset $\mathcal{A}$
**for** *generation* $= 1$ **to** $50$ **do**
    Sample batch $\mathcal{B}_1$ (size *B_size*) from $\mathcal{T}$
    Create shuffled copy $\mathcal{B}_2$ of $\mathcal{B}_1$
    **for** $i = 1$ **to** *B_size* **do**
        **Encoder Training:** Update encoder using $\mathcal{B}_1[i]$
        **Decoder Training:** Update decoder using $\mathcal{B}_2[i]$
        **for** $k = 1$ **to** $20$ **do**
            **Autoencoder Training:** Update both networks using random sample from $\mathcal{A}$
        **end**
    **end**
    **Update Roles:**
        tutor ← pupil
        pupil ← `create_Agent()`
**end**

**Algorithm 3.1: Training process for one generation.**

Training is conducted using Stochastic Gradient Descent (SGD) with a learning rate of $\eta = 5.0$, and Mean Squared Error (MSE) is used as the loss function across all training phases. As discussed earlier, the logic for forward and backward propagation is handled within the training loop, rather than being encapsulated within the agent class itself.

## 3.2 Metrics in the Semi-Supervised ILM

In this section, we demonstrate the computational application of metrics based on the theoretical explanations presented in Section 2.5.

### 3.2.1 Stability

Stability measures the extent to which two agents agree on the language they use. Specifically, it evaluates whether the meaning encoded by an agent could be successfully decoded back by another agent. We measure this once the pupil training is complete, by composing the tutor's encoder $\hat{e}_t$ with the pupil's decoder $\hat{d}_p$, or *vice versa*, to form a cross-agent autoencoder $\hat{a}_s$:

$$\hat{a}_s = \delta \circ \hat{d}_p \circ \hat{e}_t \tag{3.1}$$

where $\delta$ is a thresholding function. For each meaning $m$ in the meaning space $\mathcal{M}$, a reconstructed meaning $m'$ is obtained via $m' = \hat{a}_s(m) = \delta(\hat{d}_p(\hat{e}_t(m)))$. If reconstructed meaning exactly matches the original meaning ($m' = m$), it is counted as a successful transmission. The stability score is then defined as the proportion of meanings in $\mathcal{M}$ for which exact reconstruction is achieved (see Equation 2.10). The corresponding implementation is shown in Figure 3.1.

```
def stability(tutor, pupil, all_meanings):
    matches = 0
    total_meanings = len(all_meanings)

    with torch.no_grad():
        for meaning in all_meanings:
            m = meaning.clone().detach().float().unsqueeze(0)
            tutor_m2s_sig = tutor.m2s(m)
            pupil_s2m_mn = pupil.s2m(tutor_m2s_sig)
            original_arr = meaning.numpy()
            decoded_arr = pupil_s2m_mn.squeeze(0).numpy() > 0.5

            if np.array_equal(original_arr, decoded_arr):
                matches += 1

    stability_score = matches / total_meanings
    return stability_score
```

Figure 3.1: **Code snippet for stability measuring Python function**. The function evaluates whether each original meaning is recovered after passing through the tutor's encoder and pupil's decoder. A match is counted only if the reconstructed bit vector exactly matches the original.

### 3.2.2 Expressivity

Expressivity measures the extent to which an agent utilises the available signal space to represent meanings. This is done by quantifying the diversity of signals produced by the agent's encoder across the entire meaning space $\mathcal{M}$, and is measured for a pupil of which the training is complete. We measure this by adding the generated signal $s_n = \delta(\hat{e}_p(m_n))$ to a set, which only allows unique items without duplicates, and obtaining the proportion of unique signals on the entire signal space $\mathcal{S}$ (see Equation 2.11).

### 3.2.3 Compositionality

Compositionality measures the degree to which complex expressions are consistently constructed from smaller, meaningful components by seeing how systematically a language maps meanings to signals. Unlike other metrics that evaluates on a complete language that a pupil creates, compositionality is measured column-wise. This is to quantify the extent to which for each fact is encoded by a particular word and then average these fact-wise measures, in turn, a compositionality measure for the whole language. It is evaluated once the pupil finishes its training and is mature, but yet to transition to tutor.

Recall that a meaning is represented as a vector $m = (m_1, m_2, ..., m_i, ..., m_{n-1}, m_n)$ where each $m_i$ is a fact, and a signal as $s = (s_1, s_2, ..., s_j, ..., s_{n-1}, s_n)$ where each $s_j$ is a word. Here, the aim is to measure the extent to which whether each fact $m_i$ maps uniquely to a corresponding word $s_j$. In order to achieve this, we create two matrices to arrange all meanings and corresponding signals generated by the agent's encoder into a matrix in $n \times 2^n$ form, where each column corresponds to the $n$-th fact or word.

**(A) Meaning Matrix**

| **Index \ f** | $m_1$ | $m_2$ | $\cdots$ | $m_{n-1}$ | $m_n$ |
|---|---|---|---|---|---|
| **0** | 0 | 0 | $\cdots$ | 0 | 0 |
| **1** | 0 | 0 | $\cdots$ | 0 | 1 |
| **2** | 0 | 0 | $\cdots$ | 1 | 0 |
| $\vdots$ | $\vdots$ | $\vdots$ | $\ddots$ | $\vdots$ | $\vdots$ |
| $\mathbf{2^n - 1}$ | 1 | 1 | $\cdots$ | 1 | 1 |

**(B) Signal Matrix**

| **Index \ s** | $s_1$ | $s_2$ | $\cdots$ | $s_{n-1}$ | $s_n$ |
|---|---|---|---|---|---|
| **0** | 1 | 0 | $\cdots$ | 1 | 1 |
| **1** | 1 | 1 | $\cdots$ | 0 | 0 |
| **2** | 0 | 1 | $\cdots$ | 0 | 1 |
| $\vdots$ | $\vdots$ | $\vdots$ | $\ddots$ | $\vdots$ | $\vdots$ |
| $\mathbf{2^n - 1}$ | 0 | 1 | $\cdots$ | 1 | 1 |

Table 3.2: **Transposed representation of meaning and signal matrices**. Each row in the meaning matrix represents one of the possible $2^n$ meanings, and columns display each fact index. The signal matrix is obtained by generating the corresponding signal for all possible $m$ and storing it in a matrix. The matrices are transposed for better readability, note that in the actual code implementation the rows and columns are the opposite.

We then calculate the minimal entropy in order to quantify how well each fact is conveyed by the corresponding word. Entropy $e \in [0, 1]$ represents uncertainty, of which lower entropy closer to 0 represents

a more reliable mapping. To do this, for each fact $i$ in $i$-th row in the meaning matrix, the algorithm examines every word $j$ in the $j$-th row in signal matrix. For each pair $(i, j)$, it calculates a probability $p$ that quantifies the correlation. Hence, it is as follows:

$$p(s_j = 1|m_i) = \frac{\sum(\text{meaning\_matrix}[i,:] \times \text{signal\_matrix}[j,:])}{2^{n-1}} \tag{3.2}$$

The reason why normalisation factor is $2^{n-1}$ is because there would be exactly half of all meanings that have the $i$-th fact as 1. The entropy $h_{ij}$ is computed via a function `calculate_entrorpy` in Figure 3.2.

```
def calculate_entropy(p):
    if p <= 0 or p >= 1:
        return 0.0
    return -p * np.log2(p) - (1 - p) * np.log2(1 - p)
```

Figure 3.2: **Code snippet of Python function to calculate entropy with a given probability $p$.** $h_{ij} = 1$ means that the given fact $m_i = 1$ maps onto the word $s_j$ both '0' and '1' with equal probability, indicating uncertainty. $h_{ij} = 0$ indicates that the given fact $m_i = 1$ maps exclusively to the word $s_j = 1$ or $s_j = 0$, indicating compositionality.

For each $i$-th fact, the $j$-th word that yields the lowest entropy is selected. One potential issue that might rise is that there may exist more than one fact that maps best to the same word. This collision can compromise one-to-one mapping, which reduces overall compositionality. To mitigate this issue, if a collision is detected, only a fact with the lowest entropy is kept associated with that word, while penalising other facts by assigning the highest entropy of 1.0 to reflect uncertainty (see Figure 3.3 for an example).

| | $s_1$ | $s_2$ | $s_3$ |
|---|---|---|---|
| $m_1$ | 0.4 | 0.2 | 0.7 |
| $m_2$ | 0.3 | 0.6 | 0.4 |
| $m_3$ | 0.8 | 0.5 | 0.9 |

**(A) Initial entropy matrix**

| | $s_1$ | $s_2$ | $s_3$ |
|---|---|---|---|
| $m_1$ | 0.4 | 0.2 | 0.7 |
| $m_2$ | 0.3 | 0.6 | 0.4 |
| $m_3$ | 0.8 | 1.0 | 0.9 |

**(B) After collision resolution**

Figure 3.3: **Entropy matrix and collision resolution**. **A**: A toy example of initial entropy matrix where each cell denotes entropy $h_{ij}$ between fact $m_i$ and word $s_j$. The highlighted indicates minimum entropy values per row, showing a collision between $m_1$ and $m_3$ at $s_2$. **B**: After applying collision resolution, $m_1$ retains $s_2$ due to its lower entropy compared to $m_3$, whilst $h_{3,2}$ (highlighted in red) is penalised to discourage the conflict.

### 3.2.4 Results of Replicated Semi-Supervised ILM

We later conclude in Section 4.1, that our findings align with what the original work [8] proposed, as such we deem that our work is successful in reproducing the original Semi-Supervised ILM.

## 3.3 Baseline Implementation of β-Variational Autoencoder

### 3.3.1 Language Generation

To address the limitations posed by the simplicity of $n$-bit binary vectors, we adopt the `dSprites`[1] dataset [23] as our input space. 'dSprites' is a dataset designed as a benchmark for evaluating the disentanglement in unsupervised learning. It consists of $64 \times 64$ binary images depicting a white shape on a black background. Each image varies across five independent latent factors: shape, scale, orientation, x-position, and y-position. That is, the features are disentangled, so whether an image is a heart has no effect on whether it is rotated 90 degrees to the right or displayed at full scale.

This structured generative process enables precise control over visual variation, making dSprites particularly suitable for evaluating how well models recover the underlying generative factors. Specifically, the latent space consists of the following factors:

[1]The dSprites dataset is available at https://github.com/deepmind/dsprites-dataset (accessed March 2025).

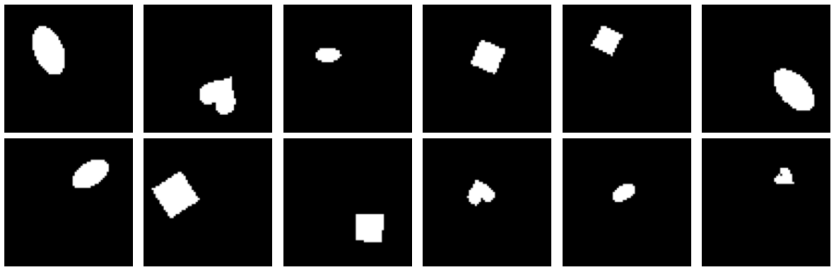


Figure 3.4: **Randomly sampled images from dSprites dataset**. Each image is generated by uniquely combining different latent features.

- `shape`: square, ellipse, heart
- `scale`: 0.5, 0.6, 0.7, 0.8, 0.9, 1.0 (6 values linearly spaced in $[0.5, 1]$)
- `orientation`: 40 evenly spaced values in $[0, 2\pi]$
- `position X`: 32 values in $[0, 1]$
- `position Y`: 32 values in $[0, 1]$

To ensure a manageable starting point for baseline evaluation, we adopt a simplified configuration that reduces the complexity faced by the VAE. Specifically, we constrain the range of each latent factor to a predefined subset, as outlined below:

- `shape`: heart, ellipse
- `orientation`: $\frac{\pi}{2}, \pi, \frac{3\pi}{2}, 2\pi$
- `scale`: $0.5, 0.6, 0.8, 1.0$

We select hearts and ellipses, as they exhibit clear visual differences across orientations. In contrast, we exclude squares that remain visually invariant under rotation within our orientation setup. Thus, the meaning space $\mathcal{M}$ comprises all unique combinations of these latent features across 1,024 positions ($32 \times 32$), resulting in 32,768 distinct images. The baseline VAE implementation serves as an upper bound for our framework, akin to a single agent learning with unrestricted access to training samples and unlimited time.

### 3.3.2 Architecture

The VAE architecture used in this project is specifically designed to address the challenges of modelling high-dimensional image data. As the input consists of $64 \times 64$ binary images, the model is adapted to efficiently handle the dense information encoded in large input vectors and to learn a compact latent space that captures meaningful and interpretable representations.

#### Encoder

In a VAE, the encoder network maps input images to latent space by outputting two vectors of a Gaussian distribution: the mean $\mu$ and variance $\log \sigma^2$. The latent variable $z$ is then stochastically sampled from $\mathcal{N}(\mu, \sigma^2)$ via reparameterization trick as detailed later in this section.

The encoder is constructed using a series of convolutional layers followed by two fully connected layers. A convolutional layer is a neural network layer that specialises in processing input data that are grid-like topology [18], hereby well-suited for processing gridded binary images from dSprites. Specifically, three sequential convolutional layers are implemented using `nn.Conv2d`, each layer halving the spatial dimension (e.g., from $64 \times 64$ to $32 \times 32$), while the number of feature maps are doubled per layer. Once the image dimension is reduced to $8 \times 8$, the output is flattened and passed to two fully connected layers, each with a hidden size of 256. Finally, a fully connected output layer produces a vector of size $2 \times d$, where $d$ is the latent dimensionality, representing the concatenated mean $\mu$ and log-variance $\log \sigma^2$ for the latent distribution. ReLU activation function is used after each layer to introduce non-linearity into the network.

**Reparameterization Trick**

With the two distributions the encoder output, we sample single latent representation $z$ using reparameterization trick, as described in Section 2.6.3. Based on Equation 2.24, we proceed the following.

```
def reparameterise(self, mean, logvar):
    logvar = torch.clamp(logvar, min=-4, max=4)
    std = torch.exp(0.5 * logvar)
    epsilon = torch.randn_like(std)
    sample = mean + std * epsilon

    return sample
```

Figure 3.5: **Code snippet for the reparameterization trick.** This function samples a latent vector from a Gaussian distribution using the mean and log-variance output by the encoder. The log-variance is clamped for numerical stability by preventing fluctuation, and the sampling is made via the reparameterization trick: $z = \mu + \sigma \cdot \epsilon$, where $\epsilon \sim \mathcal{N}(0, 1)$.

The 'sample' from Figure 3.5 represents a latent variable $z$, which differs from the original Semi-Supervised ILM in a sense that it is not strictly discretised to binary values. As a result, the signal generated and transmitted to the pupil is continuous. This can be interpreted as the communication of an internal state, akin to a thought. Despite its resemblance to thought rather than language, here we treat these continuous signals as equivalent to language for the purpose of this framework. The implications of this design choice on the model and the use of continuous vectors will be discussed in more detail in Section 3.4 and Section 5.2, respectively.

**Decoder**

Recall that the decoder in a VAE is responsible for reconstructing the original input data from the latent representation $z$. We employ a decoder that mirrors the encoder architecture by progressively upsampling the latent vector into a high-resolution image.

The decoder first passes the latent vector previously sampled using the reparameterization trick through a series of fully connected layers and expands it into a spatial feature map of shape $128 \times 8 \times 8$, which corresponds to the output of the encoder's final convolutional layer. This feature map is then successively upsampled through a series of transposed convolutional layers to restore the spatial resolution of the original image. These layers reverse the dimensionality reduction performed during encoding by progressively upsampling from $8 \times 8$ back to the original image input size $64 \times 64$. This enables the decoder to reconstruct an image from compact latent representations. Similar to the encoder, ReLU activation functions are applied between layers to introduce non-linearity. A final sigmoid activation constrains the output pixel values to the range $[0, 1]$, and ensure consistency with the scale of the original input images. The complete architecture of a VAE with the reparameterization process is illustrated in Figure 3.6.

### 3.3.3 Loss Function

During the initial phase of training a standard VAE, there was a persistent issue of posterior collapse, where the KL divergence term in the loss function diminishes to near zero. This means that the model is failing to meaningfully encode the meaning into the latent space, and instead defaulted to decoding primarily from the prior [6], resulting in blurry and uninformative reconstructions (see Figure 3.7). To address this issue, we adopt the $\beta$-VAE framework [9].

A $\beta$-VAE is a variant of the VAE that introduces a new hyperparameter $\beta$ to scale the KL divergence term. While increasing $\beta$ (i.e., $\beta > 1$) is typically used to promote disentanglement by enforcing stronger regularisation on the latent space, it can lead to posterior collapse when the decoder dominates the reconstruction objective [6]. Instead, we use a relaxed constraint on the KL divergence by setting $\beta < 1$ to allow more flexibility to diverge from the prior. Additionally, we empirically observe that with a carefully chosen value of $\beta$, the model is still able to maintain meaningful and structured latent variables. This observation will be discussed in detail in Section 4.2.

For the reconstruction loss, we use Mean Squared Error (MSE) loss to quantify the difference between the original image and the reconstructed image. A threshold $\delta = 0.01$ is set to reflect our objective not

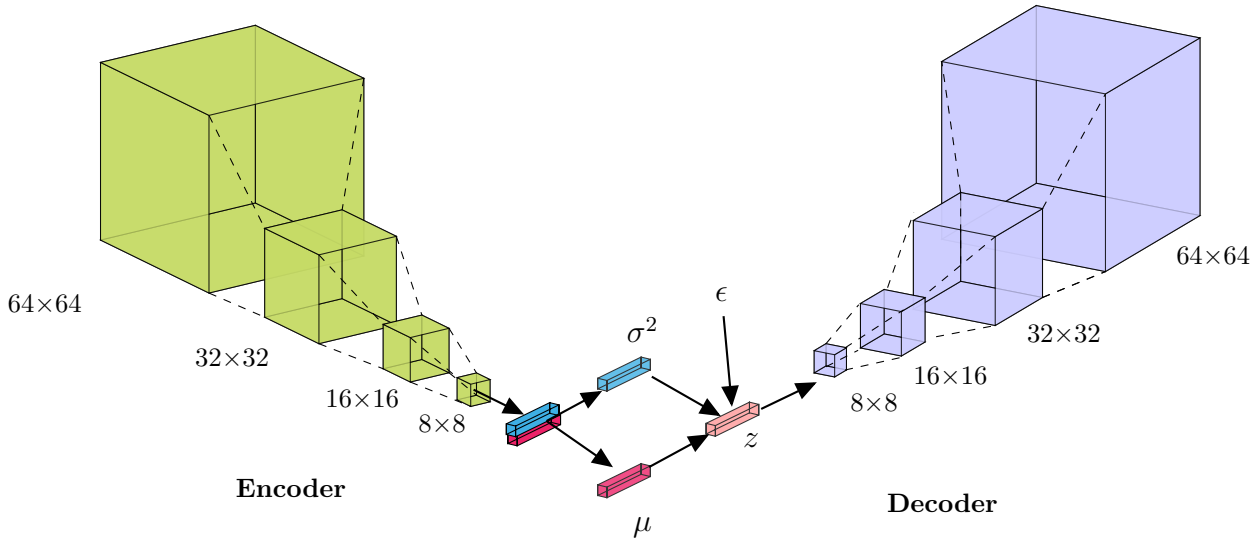



Figure 3.6: **Architecture of a Variational Autoencoder (VAE)**. The green cubes in the **encoder** represent convolutional layers built with `nn.Conv2d`. Each layer reduces the spatial dimensions of the input image by half until it reaches a compact representation of size $8 \times 8$. Note that the two fully connected layer is omitted from the diagram for simplicity. After passing through two fully connected layers, the encoder produces a vector comprising with the mean ($\mu$) and variance ($\log \sigma^2$) of the latent distribution. The region between the encoder and decoder represents the reparameterization process. The red and blue stacks represent $mu$ and $\log \sigma^2$, respectively. Using the reparameterization trick, where $\epsilon \sim \mathcal{N}(0, I)$, a latent vector $z$ is sampled as $z = \mu + \sigma \odot \epsilon$. Sampled $z$ is then passed into the **decoder**, which gradually reconstructs the image by transforming the 1D vector back into a 2D spatial structure. Like the encoder, the purple cubes represent the convolutional layer, but differ by using transposed convolutions `nn.ConvTranspose2d` instead. The decoder effectively reconstructs the image by reversing the encoder's process.

to obtain pixel-wise identical outputs, but rather to preserve the key structural features of the input and allow a controlled degree of variability. The KL divergence between the learned latent distribution and the standard normal distribution is computed as:

$$\mathcal{KL}[\mathcal{N}(\mu, \sigma^2)||\mathcal{N}(0,1)] = -\frac{1}{2}\sum(1 + \log(\sigma^2) - \mu^2 - \sigma^2) \tag{3.3}$$

The total loss is a sum of these two components, where the weight of the KL divergence is controlled by a hyperparameter $\beta$. Hence, the total loss is expressed as:

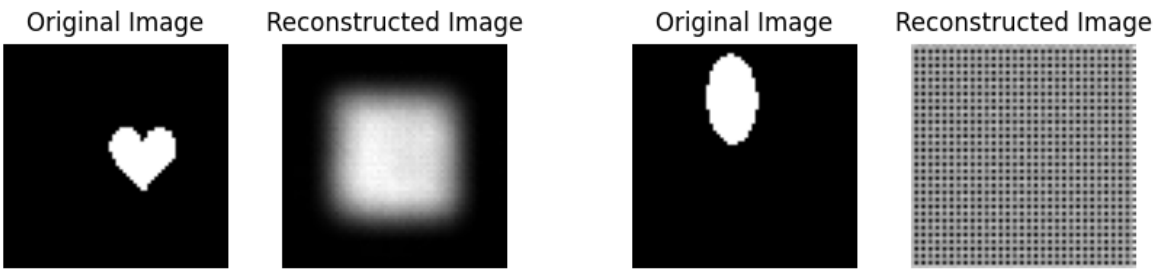
$$\mathcal{L} = \mathbb{E}_{z\sim Q(z|X)}[\log P(X|z)] - \beta \cdot \mathcal{KL}[\mathcal{N}(\mu, \sigma^2)||\mathcal{N}(0,1)] \tag{3.4}$$

### 3.3.4 Training Process

We progressively expand our dataset from two separate baseline VAEs – one for heart-shaped images and one for ellipses – with limited features to include additional characteristics. The baseline datasets consists of two orientations $\{0, \frac{\pi}{2}\}$, two scales $\{0,\ 1\}$. Figure 3.9 illustrates examples of reconstructed images by a tuned $\beta$-VAE.

After successfully training two separate VAEs, we expand the complexity of the latent space by combining the two image sets and increasing the number of options to four for each latent factor excluding shapes, which remains to be either hearts or ellipses. To facilitate this transition, we introduce a KL annealing schedule that linearly increases $\beta$ over the first 40 epochs, allowing the model to initially focus on accurate reconstruction before encouraging latent regularisation.

During training, we use the Adam optimiser with gradient clipping (maximum norm of 1.0) to ensure stable convergence. The loss function combines MSE for image convergence and scaled KL loss, as described in Section 3.3.3. After each epoch, we evaluate the model by calculating the average reconstruction loss and KL divergence on the test dataset, which comprises 20% of the entire dataset.




(a) (b)

Figure 3.7: **Example reconstruction failure using a standard VAE**. The model fails to capture latent features in the original image, leading to a blurry and uninformative reconstruction. This reflects posterior collapse, where KL divergence dominates the loss and the decoder ignores $x$. **(a)** is producing an averaged, generic output rather than capturing particular features from the latent space. **(b)** suggests that the decoder is ignoring the latent representations, producing a noisy grid image.

```
def loss_function(recon_x, x, mean, logvar, lat_dim, beta):
    recon_loss = nn.MSELoss()(recon_x, x)
    kl_loss = -0.5 * torch.mean(1 + logvar - mean.pow(2) - logvar.exp())
    kl_loss = kl_loss / (recon_x.shape[0] * lat_dim)

    return recon_loss + beta * kl_loss
```

Figure 3.8: **Python implementation of the VAE loss function.** The loss consists of a reconstruction term, computed using mean squared error between the input $x$ and its reconstruction $\hat{x}$, and a KL divergence term that penalizes deviation from the standard normal prior. The KL term is normalized by the batch size and latent dimensionality $d$, and weighted by a hyperparameter $\beta$ to allow control over the strength of the regularization.

### 3.3.5 Results of the Baseline VAE

We evaluate the model's ability to reconstruct the image by examining the training and test loss, and comparing the original image with the reconstructed image. We later conclude in Section 4.2 that our model successfully reconstructs the given images under certain hyperparameter conditions, as such we deem that this model is a trustable VAE baseline to be used in our next step of extended Semi-Supervised ILM.

## 3.4 Semi-Supervised ILM with β-VAE

### 3.4.1 Language Generation

In this setup, the meaning space is a subset of dSprites images. Similar to the Semi-Supervised ILM, a tutor is asked to generate a corresponding signal for every meaning $m \in \mathcal{M}$ to create a set of meaning-signal pairs $\mathcal{T}$. Since a VAE produces a distribution in the latent space rather than a deterministic output, we use the reparameterization trick to sample a single latent vector from the generated distribution. This could be seen as a process of a caregiver choosing one way to describe a given concept among many possible descriptions to a child. Hence, the sampled vector serves as the signal associated with the given meaning, and remains continuous rather than being discretised. Additionally, the range of latent features is identical to those used in the final baseline model from . For ease of notation, we refer to an extended model developed by this project as 'Variational Semi-Supervised Iterated Learning Model' (VS-SILM).

### 3.4.2 Agent Architecture

The agent structure is identical to the model structure described in Figure **??**. As we are simply transitioning from an autoencoder to its variant, the model preserves the fundamental encoder-decoder architecture


(a) (b)

Figure 3.9: **Example images of an original image and its reconstruction**. The VAE successfully reconstructed a slightly blurred version of the original image, capturing its overall structure and key visual features. **(a)** and **(b)** shows the reconstruction of an ellipse and a heart image, respectively.

of the Semi-Supervised ILM. Agents additionally have a `num` attribute that specifies the number of generations. One essential difference from the Semi-Supervised ILM is the addition of a reparameterization step to obtain $z$, due to the probabilistic nature of the $\beta$-VAE.

### 3.4.3 Training Process

Overall, training procedure inherits a mixture of supervised and unsupervised training approach used in the original Semi-Supervised ILM shown in Figure 2.8, with a few modification introduced to accommodate the architectural differences between standard autoencoder and $\beta$-VAE.

The process begins by generating three datasets $\mathcal{B}_1$, $\mathcal{B}_2$, and $\mathcal{A}$, which are used for unsupervised and supervised training of the agent, respectively. With each generation, the supervised dataset $\mathcal{A}$ and unsupervised datasets $\mathcal{B}_1$ and $\mathcal{B}_2$ are regenerated. Recall that tutors are asked to produce distributions and sample a latent vector, thus, we treat the sampled $z$ as a signal. Similar to the original framework, $\mathcal{B}_2$ is derived by shuffling $\mathcal{B}_1$.

The pupil's encoder and decoder are separately trained via supervised learning, using $\mathcal{B}_1$ and $\mathcal{B}_2$ as the respective training sets. During each iteration, the pupil receives a meaning-signal pair from $\mathcal{B}_1$ and $\mathcal{B}_2$, and trains its encoder and decoder accordingly. Both use MSE as a loss function to ensure the pupil's output align closely with the tutor's output.

For every supervised training step, unsupervised training is performed 20 times with sampled meanings from $\mathcal{A}$ with replacement. In contrast to supervised training, unsupervised learning trains the entire autoencoder by encoding the given meaning into distribution, sampling a latent vector $z$, reconstructing $z$, and calculating the loss against the original input. Hence, the corresponding loss is computed using the objective defined in Equation 3.4.

After the pupil is done with its training, the tutor is removed from the model and the mature pupil transitions into the role of tutor in the next generation. This process is repeated until the desired number of generation is reached.

## 3.5 Measuring the Variational Semi-Supervised ILM

Due to change in the meaning structure, which no longer aligns with the format of signals, the original XCS evaluation metrics are no longer applicable. Therefore, we introduce revised methods for measuring stability that is more plausible to this new framework.

### 3.5.1 Generating Evaluation Dataset

Before delving into the revised evaluation metric, we first define how the evaluation dataset is generated. The primary objective of this model is to evaluate whether agents can successfully learn to map visual meanings to signals, and generalise them based on what their tutor has conveyed. Majority of meanings in $\mathcal{M}$ vary in appearance due to different combinations of latent features; however, within the same combination, these variations are limited to position alone. For example, a single semantic combination of shape, scale, and orientation is instantiated across $32 \times 32$ different positions. This is different from the original Semi-Supervised ILM, where each meanings was clearly distinguishable from the others, and

there was no internal variation within a single meaning. While these positional shifts bring the variability that we seek, they do not alter the underlying meaning we are interested in, and thus it is unnecessary to include all 1,024 instances in the evaluation set. Hereby, we efficiently create a representative test set by randomly sampling 100 instances for each unique combination of latent features and avoid unnecessary duplication while preserving the semantic diversity needed for evaluation.

### 3.5.2 Stability

It is worth again noting that stability is a metric used to assess the degree of language agreement between two agents. In this context, we retain this objective and adapt the original calculation to accommodate the VAE framework, as the encoder no longer produces a single discrete output. As a result, we establish an adapted way of measuring stability, which proceeds as follows.

The tutor first encodes a given meaning into a probability distribution using its encoder $e_t$. A single latent representation $z$ is sampled from this distribution via reparameterization trick, and is then passed to the pupil's decoder $d_p$ to generate a reconstructed output. To evaluate the agreement, we compute the MSE between the tutor's original input and the pupil's reconstruction and calculate the proportion of reconstructions with lower loss than threshold $\hat{\delta}$[2]. Formally, we measure stability by:

$$\text{stability} = \frac{1}{N}\sum_{i=1}^{N}[\text{MSE}(m_i, \hat{m}_i) < \hat{\delta}] \tag{3.5}$$

where $\delta$ is a MSE threshold, and $\hat{m}_i$ is defined as:

$$\hat{m}_i = d_p(R(e_t(m_i))) \tag{3.6}$$

where $R$ represents reparameterization process. The reconstruction is considered successful and taken as an evidence that the agents agree on a shared language if the MSE value does not exceed a threshold of $\delta = 0.01$. This threshold reflects an tolerable degree of variability, consistent with the alignment objectives of the project. The stability score is then calculated as the proportion of successful reconstructions over the total number of samples.

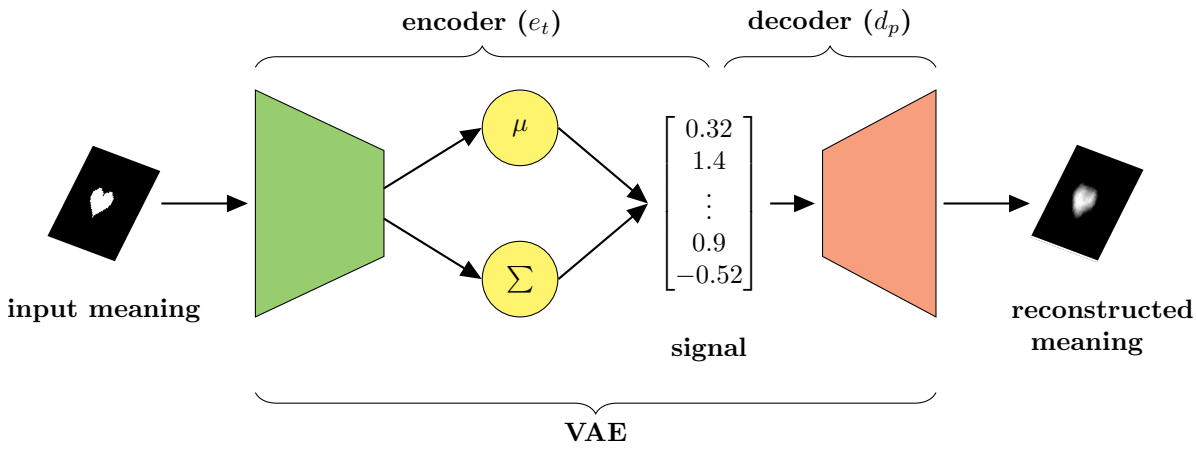



Figure 3.10: **Measuring stability via a shared VAE structure**. The tutor's encoder maps the meaning to two distribution in the latent space, $\mu$ and $\sigma$. A latent vector is sampled using the reparameterization trick and passed through the pupil's decoder to reconstruct the meaning. Due to the stochastic nature of the sampling and MSE loss function, the reconstructed output may appear slightly blurred.

This way of measuring stability is akin to concatenating one's encoder with the other's decoder, and treating them as a complete VAE as shown in Figure 3.10. Beyond its technical function, this approach offers a principled and elegant test of representational alignment: if communication succeeds under these conditions, it signals a meaningful convergence in the agents' internal languages, despite their independent learning processes.

[2]While it is common to denote the threshold as *delta*, we denote it as $\hat{d}$ to avoid confusion with the thresholding function $\delta$ used in the original Semi-Supervised ILM.

## 3.6 Software Installation

The environments and libraries used for each implementation are as follows:

- All code was written in `Python` and run on `Python 3.11.4`.
- Neural networks were trained using `PyTorch 2.5.1`.
- `numpy 1.26.4` used for efficient numerical operations and array manipulation.
- `random` was used for creating supervised and unsupervised dataset by random sampling with replacement.

# Chapter 4

# Results

## 4.1 Replication of Semi-Supervised ILM

We start by replicating[1] the Semi-Supervised ILM as described by Bunyan et al. [8]. Figure 4.1 illustrates the resulting expressivity, compositionality, and stability scores for our replication alongside those reported in the original work. As shown, all three metrics surpass the threshold $\lambda = 0.95$, thereby confirming the conclusions drawn by the original paper regarding the model's high expressivity, compositionality, and stability.

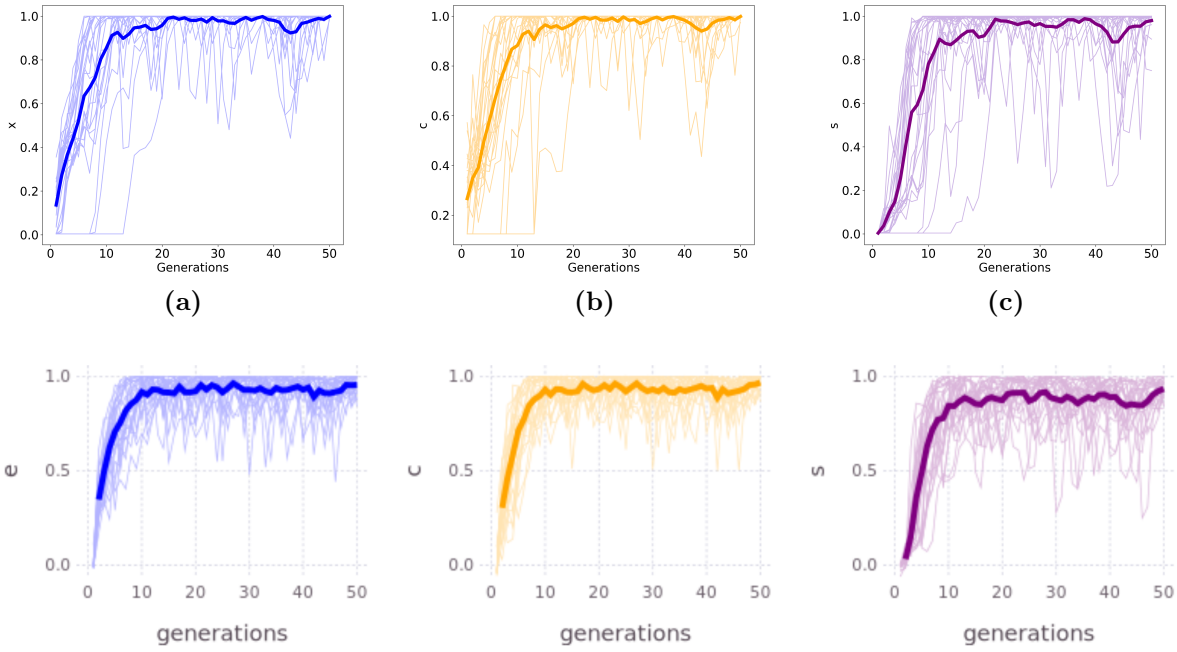



Figure 4.1: Figures **(a)-(c)** demonstrate the performance of the replicated model on expressivity (x), compositionality (c), and stability (s), respectively. These are evaluated across 50 generations within 25 independent replicates of the model. Each agent's initial state is initialised as an autoencoder with three fully connected layers and random weights. Here, $n = 8$ and thus $2^8$ meaning-signal pairs are provided, with the bottleneck set as $|\mathcal{A}| = |\mathcal{B}| = 75$. The neural network is trained using stochastic gradient descent with a learning rate of $\eta = 5.0$. Figure **(d)** shows the performance of the original Semi-Supervised ILM taken directly from [8]. We observe that the replication follows an identical trajectory to the original model.

Moreover, the trajectories of all three metrics resemble the patterns observed in the original study,

[1] The implementation code for the replication in Python can be accessed on GitHub at https://github.com/IteratedLM/2025_ilm_python.

thereby proving that the Semi-Supervised ILM consistently converges to a state in which the emergent language is stable, expressive, and compositional. This supports the model's ability to produce structured linguistic systems over successive generations of learning and transmission. As a result, this reinforces the fidelity of our replication and supports the conclusions drawn by Bunyan et al. [8], confirming that our model is a faithful recreation. Accordingly, we regard this implementation as a reliable baseline for building and experimenting with an extended version of the Semi-Supervised ILM.

## 4.2 Baseline Variational Autoencoder

### 4.2.1 Single Shape Configuration

To build an extended version of Semi-Supervised ILM, we first start with a baseline VAE trained under simplified meaning space. Specifically, the total number of latent dimensions is fixed, and each latent feature is constrained to represent a limited set of possible values. Once the VAE successfully learned to reconstruct data under these restricted conditions, we progressively increased the complexity by expanding the range of values for each latent feature to encourage the model to develop more complex, structured latent representations.

We constructed two separate VAEs for ellipse and heart dataset. As a careful balance between reconstruction loss and KL divergence loss, we found an optimal $\beta$ value for each model through empirical approach.

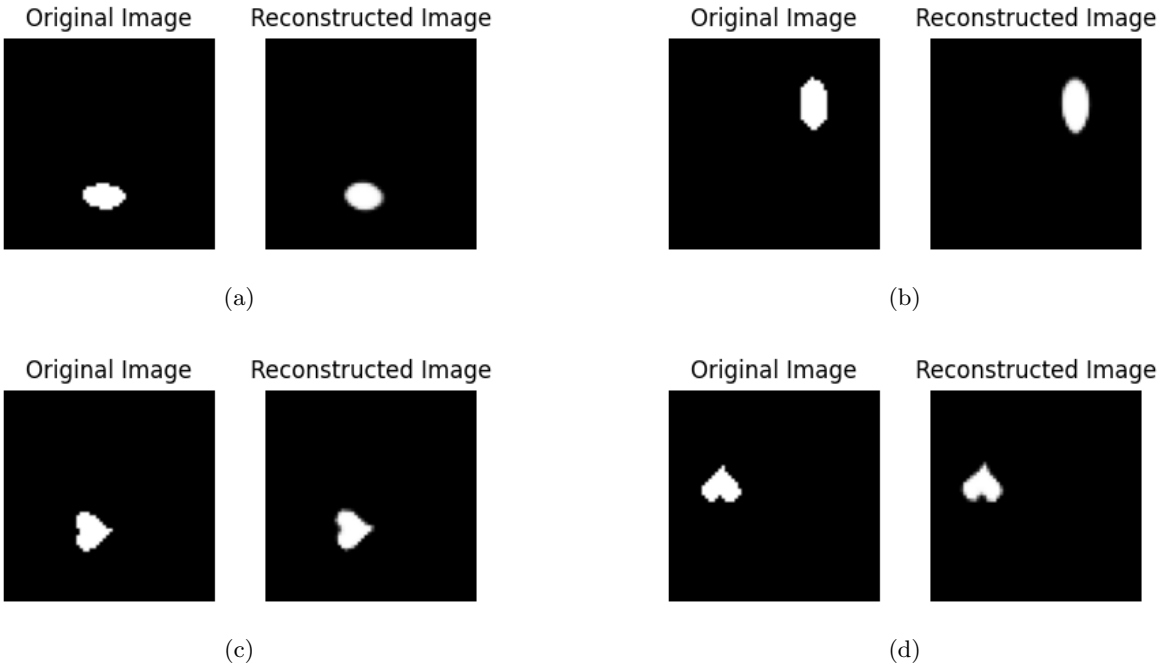
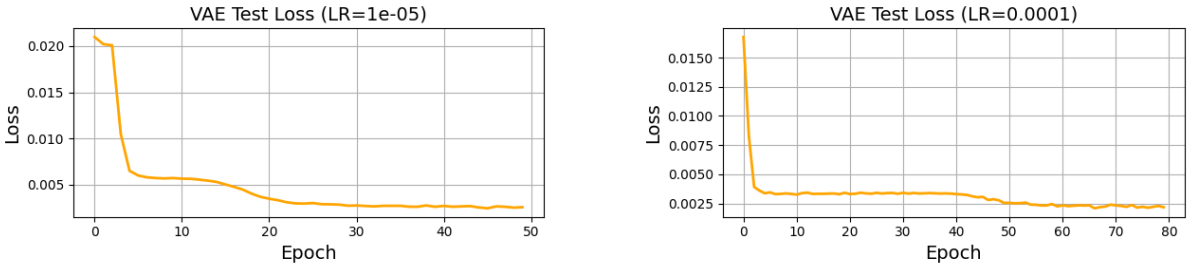



Figure 4.2: **Reconstruction results from the two separate vanilla VAEs. (a) and (b)** show reconstructed ellipse samples with varying scales and orientations, while **(c) and (d)** are of reconstructed heart images. In both cases, the model successfully reconstructs the original image of the underlying structures. Minor blurring observed along is attributed to the use of MSE loss function.

The results presented in Figure 4.3 demonstrates that, after a certain amount of epoch, both VAE models are able to generate plausible reconstructions. The optimal $\beta$ value was found to be $\beta = 0.00001$ for the ellipse model and $\beta = 0.0001$ for the heart model. This is expected, as heart images are more complex compared to ellipses and requires a stronger regularisation to encourage the model to develop a more structured and expressive latent representation. Hence, the total loss per models are shown in Figure 4.3.

Based on the plausible reconstruction results shown in Figure 4.2, we conclude that a valid baseline for the $\beta$-VAE has been successfully established on the dSprites dataset. These results demonstrate the effectiveness of our implementation in learning meaningful representations and producing accurate reconstructions, thereby confirming the viability of our model as a functional VAE for this dataset.

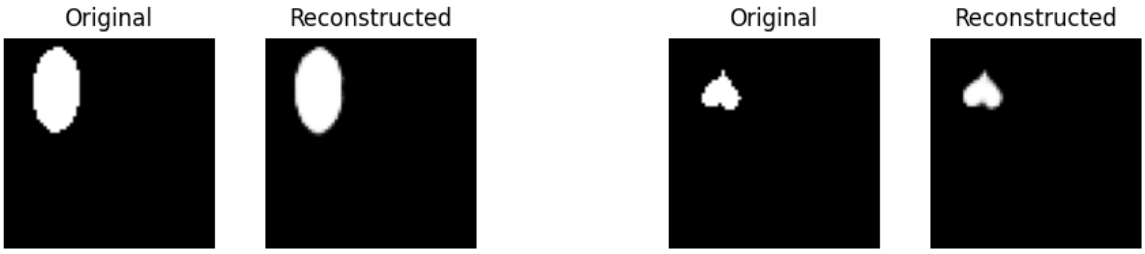



(a) **Test loss on the ellipse dataset.** (b) **Test loss on the heart dataset.**

Figure 4.3: **Comparison of test losses across datasets.** The performance of the model is shown for the ellipse (left) and heart (right) datasets.

### 4.2.2 Increasing Feature Value Granularity

We now expand the feature value granularity from previous dataset, to 4 orientations and 4 scales. This greatly increases the size of $\mathcal{M}$ from 8 possible latent feature combinations to 32. The expectation was that since it needs to learn more latent features than the previous version of the VAE, it would require more time to pick up latent features and learn how to map each characteristic of the image to a particular latent feature in the signal space. Figure 4.5 shows the trajectory of total loss. Interestingly, the model was able to achieve a low loss value almost from the very first stage of the epoch, different from our initial assumption. Yet, it was capable to reconstruct the given image properly as stated in Figure 4.4.




(a) (b)

Figure 4.4: **Reconstruction samples with increased size of meaning space**. The meaning space is expanded 2 shapes, 4 orientations, and 4 scales. The $\beta$-VAE successfully reconstructs the given image.

From this, we conclude that from Figure 4.4 and Figure 4.5 that the VAE we constructed is a working VAE that is capable of capturing meaningful features from the latent space. Given its low loss on test set, we confirm that in the early epochs, the VAE is capable of learning the latent features, and to regenerate the given meaning by reconstructing the features from a encoded signal.

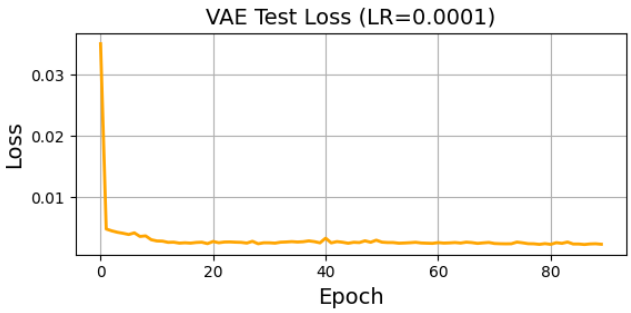



Figure 4.5: **Test loss over epochs on expanded meaning space**. $\eta$=0.0001, $\beta$=0.0001. The model achieves remarkably low loss from the early stage.

## 4.3 Variational Semi-Supervised ILM

We build a VS-SILM that extends the Semi-Supervised ILM by incorporating a $\beta$-VAE in a place of the standard autoencoder for each agent. The modifications enables us to explore the following key research questions, which are examined through the experimental findings presented in the subsequent sections:

- Given an appropriate bottleneck size of meanings, are the agents in the new model capable of generating a fully stable language (i.e., achieving a stability score higher than $\lambda = 0.95$)?
- How does $\beta$-regulated latent learning shape the stability of emergent language?
- Where full stability or improvements are observed, what underlying structural or learning-related factors contribute to this outcome?

Notably, while the transition from a standard autoencoder to a $\beta$-VAE may make the agent's internal representation more realistic and disentangled, it does not necessarily guarantee that both agents will map the same facts to same words. Hence, language requires shared conventions, not just well-organised internal features. Experiments show that the $\beta$-VAE fails to learn a shared language that is consistently agreed upon between agents. Nevertheless, under specific conditions of bottleneck size and number of supervised training epochs, a limited degree of language stability is achieved. This, however, does not necessarily imply that the agents have developed a shared or compositional language; rather, it may reflect that agent is individually learning mappings without establishing a stable linguistic convention. These are discussed further in Chapter 5.

### 4.3.1 The Effect of Supervised Epochs on Stability

From this point onward, we define the state of a language whose stability exceeds the threshold ($\lambda = 0.95$), thereby satisfying the stability requirement for an XCS language, as 'fully stable'. The number of supervised training epochs represent the length of time of which a pupil is trained to acquire a language. The expectation is that longer the time given to a pupil for training, either it succeeds in reaching a stable language, or even if it does not, it shows some language agreement between some series of generations.

Additionally, we set the meaning space $\mathcal{M}_1$, that consists of 2 shapes (i.e., ellipse and heart), 4 orientations, and 4 scales.

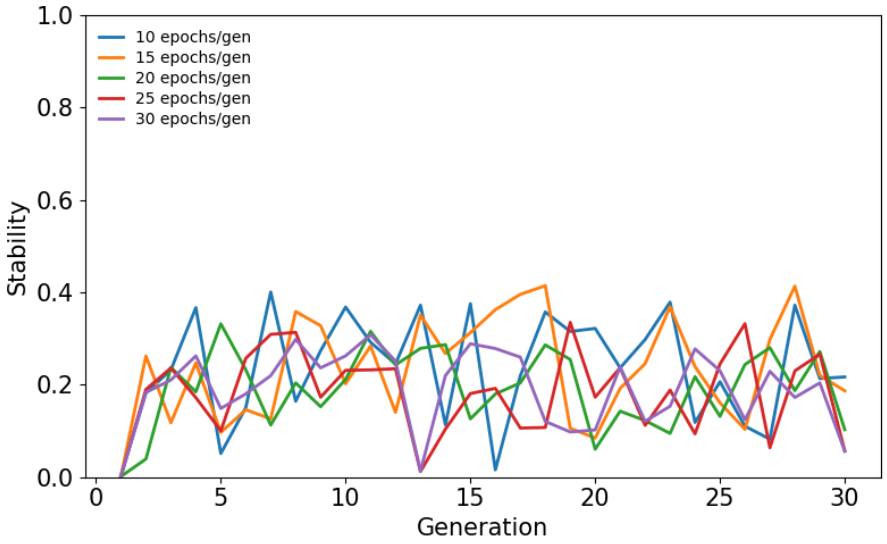



Figure 4.6: **Stability score across 30 generations that each varies in number of epoch**. In all cases, $|\mathcal{B}| = 200$ and $\beta = 0.0001$. Their results showed that across the entire generation, under no conditions agents were able to evolve a stable language.

As shown in Figure 4.6, the stability does not exhibit a continuous increase or decrease but instead oscillates across generations within a very small range that rarely exceeds 0.4. This suggest either that the current bottleneck is too small for the agents to be exposed to all unique latent feature combinations,

or that agents are not given enough time to fully learn the mapping between meanings and signals. While increasing either the bottleneck size or training time could possibly mitigate this issue, due to computational constraints and time limitations, we instead reduce the size of $\mathcal{M}_1$ by decreasing the number of available options per latent feature into two. This limitation is further addressed in Chapter 5.

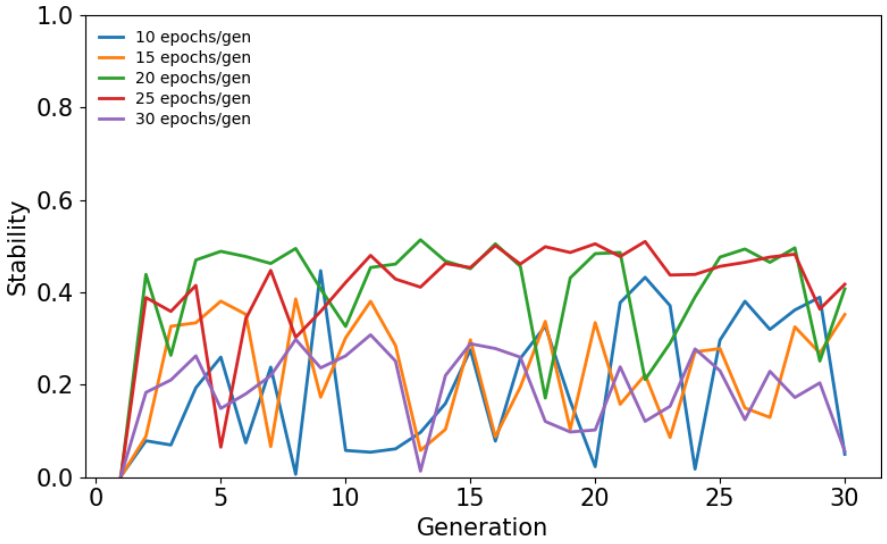



Figure 4.7: **Stability score across 30 generations with varying number of epochs and smaller meaning space**. $|\mathcal{B}|$ is again 200 in all cases, and $\beta = 0.0001$. Even in this setup, the agents fail to evolve a fully stable language.

Similar to the previous setup, stability remains volatile across generations, but we observe that the range of oscillations and consistency improves when agents are trained with a larger number of epochs. This may suggest that extending the language training duration enhances the agents' capacity to generalise unseen meaning-signal mappings to some extent. Crucially, despite the reduced size of $\mathcal{M}$, a fully stable language did not emerge under any experimental condition, as shown in Figure 4.7.

It is notable that the model trained for $t = 25$ epochs exhibited the highest peak stability of 0.514 and maintained relatively consistent performance from the eighth generation to the final generation. Although its stability fluctuated somewhat, dropping to approximately 0.4 in later generations, this simulation achieved the most balanced language in terms of both consistency and performance across the transmission process.

One particularly interesting finding is that extending the number of training epochs beyond 25 to $t = 30$ did not further improve stability. In fact, it showed a marked decline in performance, with lower stability scores and persistent volatility throughout the generations. This decline in performance with extended training likely results from an overfitting effect, where the pupil agent overlearns the specific patterns of its tutor, including both the systematic features and the noise or irregularities in the tutor's outputs. When this pupil subsequently becomes a tutor, these amplified irregularities are passed on to the next generation, potentially leading to cumulative degradation of the language structure across generations.

Hence we conclude that delicate balance is required in iterated learning systems: too little training fails to establish robust language features, while too much training may increase fidelity to a specific teacher but reduce the generalisability and stability of the transmitted language.

### 4.3.2 The Effect of Transmission Bottleneck Sizes on Stability

The size of transmission bottleneck is a critical parameter in emergent language models. If it is too small, the language fails to stabilise. If it is too large, agents may receive enough meaning-signal pair examples to learn the language independently of their tutor, ending up with non-expressive and non-compositional language. Here, we assess whether an optimal bottleneck size exists, as it determines the conditions under which language can evolve into a stable and structured form, as was observed in the Semi-Supervised ILM. If full stability is not achieved within the explored range of bottleneck sizes, we further examine the extent to which bottleneck size influences the trajectory of language stability over generations. Based on

the findings from the Semi-Supervised ILM, we begin under the assumption that appropriate bottleneck conditions exist to support continuous agreement of language transmitted across generations.

Compared to the proportion of transmission bottleneck relative to the entire meaning space in Semi-Supervised ILM, the bottleneck in VS-SILM is extremely small and may contain numerous duplicate combinations of latent features that differ only in their positions. Following [8], the optimal bottleneck capacity for XCS language emergence scale linearly with meaning size. In the case of a $64 \times 64$ meaning space, an optimal bottleneck would correspond to an estimated 42,996 meaning-signal pairs. However, due to the computational constraints and time limitations, the experimental bottleneck range would be restricted to $[200, 400]$, representing less than 1% of the full meaning space. Limitations stemming from this approach are discussed in greater detail in Section 5.4.

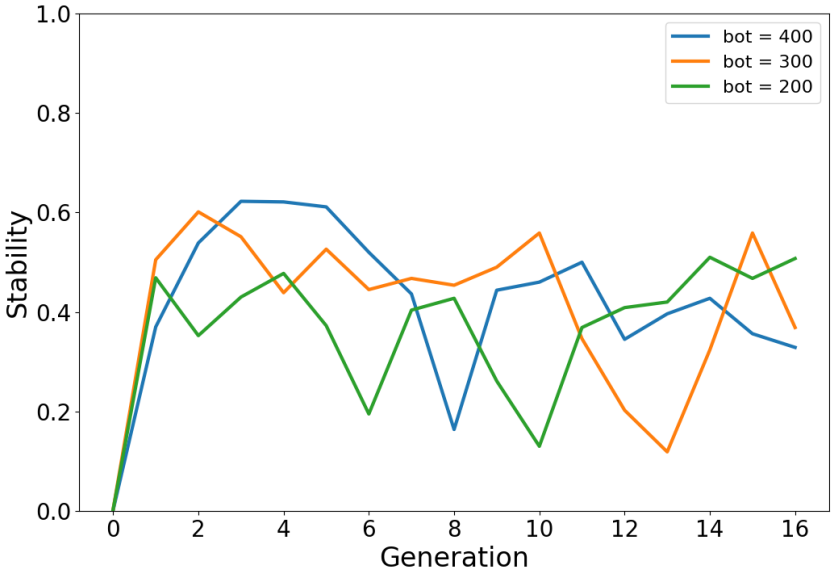



Figure 4.8: **Stability measures across 16 generations that varies in bottleneck sizes**. In all cases, epochs were at 25 and $\beta = 0.0001$. The result shows the agents were not able to evolve a fully stable language under any conditions.

Figure 4.8 demonstrates that none of the conditions exhibited the emergence of a fully stable language. However, partial stability did emerge between agents, albeit with several oscillations over time.

The largest bottleneck condition, $|\mathcal{B}| = 400$, achieves the highest maximum stability (0.64) among all simulations, demonstrating rapid early agreement. Nevertheless, it fails to maintain consistent stability, and shows only temporary convergence between agents, notably between generations 1–4 and 9–11.

The smallest bottleneck condition of $|\mathcal{B}| = 200$ begins with lower initial stability (0.46), but exhibits a notable recovery toward the later generations, ultimately achieving its highest stability by the end of the simulation. The intermediate bottleneck condition, $|\mathcal{B}| = 300$, performs similarly to the largest bottleneck early on ($s = 0.62$), but experiences a substantial decline and subsequent fluctuations.

Given the severely limited bottleneck sizes, it is likely that the observed stability results from agents memorising a small set of meaning-signal mappings, rather than learning meaningful, compositional structures in latent space. Without a direct measure of compositionality in VS-SILM, it remains unclear whether the agents are genuinely developing structured language or simply overfitting (i.e., memorising) to the limited information available. However, considering the excessive bottleneck restrictions, memorisation appears to be the more probable explanation.

Based on these findings, we conclude that under the current bottleneck conditions, full language stability does not emerge. Hence, the tested bottleneck range is not optimal for supporting the evolution of stable and structured language.

### 4.3.3 The Effect of Beta ($\beta$) on Stability

Recall from Section 3.3.3 that $\beta$ controls the regularisation strength by weighting KL-divergence term. A small $\beta$ applies weak regularisation and encourages the model to prioritise accurate reconstruction

over latent space structure. Conversely, a large $\beta$ imposes strong regularisation to force the model to encourage plausible latent representations. Albeit it is somewhat unnatural that introducing $\beta$ makes the model's performance to rely on formulating very specific loss function, it benefits from giving a nice trade-off between reconstruction loss and KL-divergence term. Hence, it is important to find an optimal value of $\beta$ that encourages structured latent representations, but also ensure the reconstructions to be accurate and efficiently represent properties of the data. We expect the existence of an optimal value of $\beta$ that serve as a threshold for overall language performance, where beyond that value the agent will focus too much on learning latent space and end up failing to create a plausible reconstruction of the given meaning. Therefore, we examine how different values of $\beta$ affect stability of the emergent language across generations.

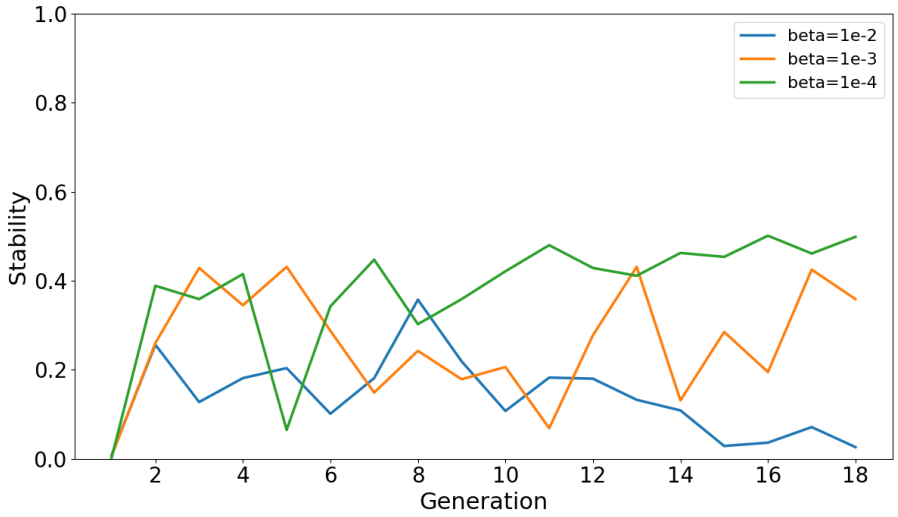



Figure 4.9: **Stability evaluation by different beta values with limited dataset**. The $|\mathcal{B}|$ is 200 and the number of epochs is 25. All conditions fail to achieve fully stable language, albeit $\beta = 0.0001$ simulation shows a consistent, gradually increasing stability trend.

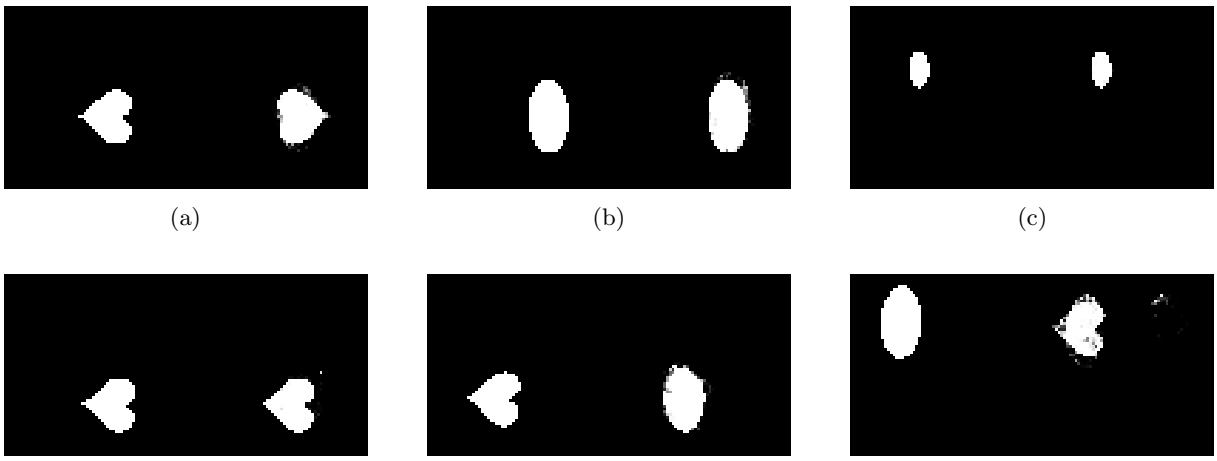
Figure 4.9 presents stability scores across 18 generations for three different simulations with varying in $\beta$ values. Across all cases, no simulation produced a fully stable language.

For the highest $\beta$ value (0.01), stability steadily declines from 0.37 to 0.02, directly contradicting theoretical expectations that structure should emerge and persist over time. This suggests that excessive emphasis on latent feature learning via KL-divergence may hinder the development of consistent language. The intermediate $\beta$ value (0.001) exhibits oscillatory stability within the range [0.14,0.43], indicating insufficient language stabilization under this parametric configuration.

Interestingly, the lowest $\beta$ value (0.0001) shows a consistent upward trajectory, with stability improving from 0.34 to 0.5 Although this still falls short of full stability, the steady increase aligns with expectations for structured language emergence and indicates that limiting latent pressure in favour of accurate reconstruction supports more stable communication.

Figure 4.10 further reveals that even with the highest-performing $\beta$ value, agents occasionally fail to correctly identify one of the latent features. This indicates that with this $\beta$ value, agents are successfully learning from the latent space to some extent and can categorise between various features. However, they sometimes confuse certain latent features and may interpret them incorrectly, resulting in misaligned representations. This specific limitation appears to be the constraining factor preventing further improvement in overall language stability.

As a result, among the tested conditions, $\beta = 0.0001$ yields the most promising balance between reconstruction loss and KL-divergence term. Higher $\beta$ values appear to over-prioritise abstract representation at the cost of communicative fidelity. Since the range of experimented $\beta$ is limited, it remains unclear whether $\beta$=0.0001 is an optimal value or it simply performed best among tested simulations. Nevertheless, these results suggest that lower $\beta$ values may support better stability in emergent language. Hence, we conclude that while no fully stable language emerged in this setup, we have identified a $\beta$ value that produces consistent and progressively improving stability performance.



(a) (b) (c)

(d) (e) (f)

Figure 4.10: **Examples of original image and reconstruction pairs (Left: input image, Right: reconstructed image)**. These samples are created by the mature pupil in the last generation under $\beta$=0.0001, $t$=25, and $|\mathcal{B}|$=200. While some successfully reconstructs the given meaning, there are occasions where one latent features are misrepresented.

# Chapter 5

# Discussion

## 5.1 Results

Our findings in Chapter 4 demonstrate successful replication of the original results using 8-bit Semi-Supervised ILM. However, when extending the challenge to more complex imagery meaning spaces and replacing standard autoencoders with $\beta$-VAEs, our work did not identify conditions that produce full stability in the emergent language.

While the original Semi-Supervised ILM did not describe performance as a function of training epochs, our work demonstrates the existence of an optimal value that serves as a threshold. Training below this threshold provides insufficient time for agents to acquire language, whereas exceeding it offers no additional benefit and may reduce generalisability.

The bottleneck size demonstrates a partially analogous impact to that observed in the original Semi-Supervised ILM. We have shown that when the bottleneck is sufficiently small, agents tend to memorise specific meaning-signal pairs rather than learning the underlying structure of language.

Our result from varying $\beta$ indicate an optimal value exists. Values above this threshold cause agents to overemphasise latent structures at the expense of accurate reconstruction. Model using the highest-performing $\beta$ successfully recreate portions of given meanings, while mismatching certain latent features in other reconstructions.

Overall, we observed the variation of hyperparameters we have explored has not resulted in a complete full stability of emergent language. Nevertheless, although transitioning to a larger meaning space introduced additional uncertainty and variability into the model, our experiments nonetheless yielded several notable findings that may offer insight into the behaviours of an ILM with variational autoencoders in emergent language settings.

## 5.2 Implications

This study shows several notable implications for understanding how language emerges when variability and uncertainty are incorporated into computational models of language evolution. By transitioning from deterministic to probabilistic representations and incorporating visual meanings, our work sheds new light on fundamental questions about language transmission and acquisition.

### 5.2.1 Bottleneck Theory

One of our most interesting findings is a nuanced relationship between transmission bottlenecks and language generalisation. Prior research suggested that smaller bottlenecks naturally lead to better generalisation and forcing agents to infer patterns across generations [30, 42, 31]. However, our empirical results challenge this assumption. Specifically, we demonstrate that excessively tight bottlenecks lead to language agreement only within constrained generation subgroups, and fail to achieve meaningful generalisation across entire generations. This suggests that rather than collaboratively developing shared abstractions across generations, agents trained under severely narrow bottleneck constraints may be memorising limited meaning-signal pairs they encounter, and constructing their own interpretation frameworks that degrade rapidly after periodic agreement within localised agent groups.

This is particularly interesting as it underscores a critical distinction between within-generation and cross-generation generalisation in language modelling. Generalisation often involves identifying patterns

from samples; essentially, updating beliefs based on the observed data [20]. The smaller the set of examples encountered by agents, the lower the contextual diversity the learners have, and it is not enough to develop robust abstractions beyond what is observed. When agents encounter extremely limited examples that lack contextual diversity, agents may generally find it insufficient to develop robust abstractions beyond what is directly observed. As a result, instead of developing productive abstractions, agents revert to high-fidelity reproduction through rote learning, resulting in temporary micro-agreements that fail to persist across generational boundaries.

This finding questions the assumption that restrictive bottleneck universally drive generalisation. Interestingly, overly constrained transmission conditions may actively impede the emergence of stability across generations. Our results suggest that optimal cultural transmission requires a delicate balance between constraint and flexibility. This work opens crucial questions about the necessary conditions for sustainable cultural transmission and the emergence of truly generative linguistic systems.

### 5.2.2 Critical Period

As mentioned earlier, there has long been a debate over the existence of a 'critical period' for language acquisition [35, 5, 13], and Iterated Learning Models (ILMs) are often built on the assumption that such a period exists [27]. Our results in Section 4.3.1 offer empirical support for this assumption. Specifically, we find that beyond a certain training epoch, additional exposure destabilises the language, suggesting that overtraining leads to diminishing returns.

This aligns with maturational constraint theories [36], which propose that language learning capacity declines over time. Although our agents do not undergo biological maturation, the observed sensitivity to exposure implies that similar functional constraints may arise purely from the dynamics of learning. While alternative explanations remain possible, the emergent behaviours in our computational models support the idea that temporal constraints play a meaningful role in shaping language acquisition and transmission.

### 5.2.3 Thoughts and Language

Our use of continuous latent vectors as signals rather than discretising them raises questions about the boundary between internal representation and externalised language. Although we frame them as language, using continuous vectors makes our model more akin to a model of thought than of language. Language and thought may be related through having structural features of language, including the ability to use words or to combine them as strings [19]. While we treat the continuous vector as a signal, the real challenge lies in encoding private mental states into shared, interpretable symbols.

The disconnection between reconstruction accuracy and language stability we observe further emphasises this distinction. Accurate reconstruction alone does not guarantee stability; rather, it emerges only when agents agree on consistent meaning-signal mappings. This suggests that successful language is not merely about representing internal states, but about reaching a shared understanding of those representations across individuals through shared conventions.

These findings suggest that the emergence of language may depend as much on coordination and social alignment as on representational fidelity. In doing so, our model offers a computational perspective on how private cognitive structures evolve into public linguistic norms; a process central to the development of human communication.

## 5.3 Future Work

### 5.3.1 Thoughts to Languages

The way that VS-SILM handles a signal differs from the original Semi-Supervised ILM. In the original model, the continuous vector output by the agent's encoder is discretised by mapping its values to either 0 or 1 based on the predefined threshold $\delta$, thereby pushing the signal to be in a binary symbolic form. In contrast, the VS-SILM introduced here bypasses such discretisation process and uses continuous vectors directly throughout the pupil's training. Although we have been referring to this as a modelling of language, what is being transmitted from a tutor to a pupil is continuous, and thus might be considered more as a communication of the internal state, or analogue of thought.

As the ultimate goal is to model a language, further research could aim to bridge this gap by introducing a discretisation process that converts the continuous latent vectors into symbolic representations.

A straightforward approach would be to reintroduce the threshold-based discretisation mechanism from the original Semi-Supervised ILM, optimising the threshold parameter to preserve critical information during the conversion. It would also be interesting to investigate how different discretisation strategies affect the fidelity of transmitted signal and the overall performance of the emerged language.

### 5.3.2 Measuring Compositionality

Distinguishing between rudimentary language systems and true, structured language requires evaluation through expressivity and compositionality [15]. Evaluating a language's expressivity in VS-SILM is relatively straightforward; we can simply count how many unique signals are used to express a given set of meanings. However, measuring compositionality poses significant challenges when meanings and signals are no longer of identical size. When meanings become larger than their corresponding signals in their sizes, we inevitably encounter cases where multiple facts map to duplicate words. This complicates traditional compositionality metrics, which typically assume clear, decomposable relationships between facts and words. Hence, developing new methodologies to quantify compositionality in such asymmetric systems would be an important direction for future research. These approaches would need to account for how complex meanings are compressed into more compact signals while still preserving compositional structure.

### 5.3.3 Richer Meaning Space

The VS-SILM transitions the meaning space from $n$-bit binary vectors to 2D greyscale images. While this transition adds a layer of complexity to the meaning space, future works could explore how increasing the richness may further influence emergence of compositional language. Examples of doing such include extending to colour images or transitioning from 2D to 3D representations. One assumption we could make is that greater complexity would encourage agents to develop more compositional language, as agents would need to combine more features when constructing utterances.

However, complexity alone may be insufficient for true compositionality to rise. Without proper constraints, agents might resort to memorisation rather than developing a structured, compositional structures. The critical factor to consider would be whether agents face pressure to generalise across seen combinations of latent features. Hence, for future work, we propose not only increasing the meaning complexity but also carefully designing meaning spaces with clearly disentangled features and structured learning environments that require generalisation.

### 5.3.4 Communities

By inheriting tutor-pupil structure from the original Semi-Supervised ILM, VS-SILM severely limits the diversity of interactions available to the learner. This, however, is not how humans learn language in reality; instead, we acquire language via rich social interaction [44]. Human learners learn through engagement of interactions with multiple tutors, pupils (e.g., talking with their peers), or even with overhearing external audio from various media sources [25, 33]. Exposure to diverse communicative agents drives the emergence of more adaptive and compositional language skills. Hence, models of language development could move beyond rigid tutor–pupil frameworks and instead support community-based learning, where agents interact dynamically and learn from one another across roles.

## 5.4 Limitations

One drawback of this study is the reliance on single-run experiments due to computational and time limitations, which constrains the statistical validity of our findings. Without having multiple runs, any observed effects could be entirely attributable to random variable or specific initial conditions rather than the mechanism being investigated. Additionally, we use a continuous signal as 'language', when it is more akin to 'thoughts'. Furthermore, we examined only a subset of potentially influential variables while holding constant critical factors including learning rate schedules, network architectures, and number of generations. Although preliminary exploration guided our parameter selection, a comprehensive sensitivity analysis would substantively strengthen the robustness of our conclusions.

# Chapter 6

# Conclusion

The prior ILM variants employ rigid, artificial frameworks that fail to account for the variable and nuanced nature of human cognition, instead relying on deterministic $n$-bit binary meaning representations. While the Semi-Supervised ILM proposed by Bunyan et al. [8] introduces a novel approach to simulate language transmission with lower computational costs through autoencoders, several limitations persist. Nevertheless, these models remain valuable for demonstrating that even with new training algorithms and architectures designed to mitigate obversion, XCS languages consistently emerge.

This study successfully replicates the Semi-Supervised ILM framework and confirms Bunyan et al.'s [8] findings that XCS languages still emerge under specific parameter conditions. By transitioning from discrete $n$-bit meaning spaces to continuous $64 \times 64$ pictorial representation, and critically, substituting standard autoencoders with $\beta$-VAEs, our model faces challenges in generating stable emergent language under different number of training epochs, transmission bottleneck sizes, and $\beta$ conditions. Despite no stable language emerging under the tested hyperparameter conditions, evidence of language acquisition suggest in the experimented conditions could potentially yield stable, expressive, and compositional language. Our findings indicate that while moderate bottlenecks may encourage generalisation across language. We suggest that while moderate bottlenecks may encourage generalisation across language, excessively narrow bottlenecks may lead agents to memorise specific meaning-signal mappings by constructing idiosyncratic representations.

From this, we propose new directions for Semi-Supervised ILM research involving the optimisation of $\beta$-VAE based agents in imagery meaning spaces. Once a plausible compositionality metric is constructed, exploring the properties of emergent language under different hyperparameters would be an interesting next step.

# Bibliography


[1] N. Akhtar. The robustness of learning through overhearing. *Developmental Science*, 8(2):199–209, 2005. doi:10.1111/j.1467-7687.2005.00406.x.

[2] N. Akhtar, J. Jipson, and M. A. Callanan. Learning words through overhearing. *Child Development*, 72(2):416–430, 2001. doi:10.1111/1467-8624.00287.

[3] D. Bank, N. Koenigstein, and R. Giryes. Autoencoders. 2021. URL: https://arxiv.org/abs/2003.05991, arXiv:2003.05991.

[4] L. W. Barsalou. Perceptual symbol systems. *Behavioral and Brain Sciences*, 22(4):577–660, 1999.

[5] D. Birdsong. *Second Language Acquisition and the Critical Period Hypothesis*. Routledge, 1999.

[6] S. R. Bowman Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz, and S. Bengio. Generating sentences from a continuous space, 2016. URL: https://arxiv.org/abs/1511.06349, arXiv:1511.06349.

[7] S. Bullock and C. Houghton. Modeling language contact with the iterated learning model. In *Artificial Life Conference Proceedings 36*, volume 2024, page 54. MIT Press, 2024.

[8] J. Bunyan, S. Bullock, and C. Houghton. An iterated learning model of language change that mixes supervised and unsupervised learning. 2024. arXiv:2405.20818.

[9] C. P. Burgess, I. Higgins, A. Pal, L. Matthey, N. Watters, G. Desjardins, and A. Lerchner. Understanding disentangling in $\beta$-vae, 2018. arXiv:1804.03599, doi:10.48550/arXiv.1801.06146.

[10] M. A. Callanan, N. Akhtar, and L. Sussman. Learning words from labeling and directive speech. *First Language*, 34(5):450–461, 2014. doi:10.1177/0142723714553517.

[11] N. Chomsky. *Knowledge of Language: Its Nature, Origins, and Use*. Praeger, 1986.

[12] N. Chomsky. *New Horizons in the Study of Language and Mind*. Cambridge University Press, 2000.

[13] R. M. DeKeyser. The robustness of critical period effects in second language acquisition. *Studies in Second Language Acquisition*, 22(4):499–533, 2000.

[14] C. Doersch. Tutorial on variational autoencoders. *arXiv preprint arXiv:1606.05908*, 2016.

[15] G. Frege. On sense and reference. *Zeitschrift für Philosophie und Philosophische Kritik*, 100:25–50, 1892. Originally published in German as "Über Sinn und Bedeutung".

[16] T. Fögen. 216animal communication. In *The Oxford Handbook of Animals in Classical Thought and Life*. Oxford University Press, 08 2014. doi:10.1093/oxfordhb/9780199589425.013.013.

[17] L. R. Gleitman, E. L. Newport, and H. Gleitman. The current status of the motherese hypothesis. *Journal of Child Language*, 11(1):43–79, 1984. doi:10.1017/S0305000900005584.

[18] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[19] A. Gopnik and A. N. Meltzoff. Language and thought. In *Words, Thoughts, and Theories*. The MIT Press, 01 1997. arXiv:https://direct.mit.edu/book/chapter-pdf/2451127/9780262274098_ccm.pdf, doi:10.7551/mitpress/7289.003.0012.

[20] T. L. Griffiths and M. L. Kalish. Language evolution by iterated learning with bayesian agents. *Cognitive Science*, 31(3):441–480, 2007.

[21] H. Gweon, J. B. Tenenbaum, and L. E. Schulz. Infants consider both the sample and the sampling process in inductive generalization. *Proceedings of the National Academy of Sciences*, 107(20):9066–9071, 2010.

[22] M. D. Hauser, N. Chomsky, and W. T. Fitch. The faculty of language: What is it, who has it, and how did it evolve? *Science*, 298(5598):1569–1579, 2002. doi:10.1126/science.298.5598.1569.

[23] I. Higgins, L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinick, S. Mohamed, and A. Lerchner. dsprites: Disentanglement testing sprites dataset. https://github.com/deepmind/dsprites-dataset, 2017. DeepMind.

[24] C. F. Hockett. The origin of speech. *Scientific American*, 203(3):88–96, 1960. doi:10.1038/scientificamerican0960-88.

[25] E. Hoff. How social contexts support and shape language development. *Developmental review*, 26(1):55–88, 2006.

[26] J. R. Hurford. *Nativist and functional explanations in language acquisition*, pages 85–136. De Gruyter Mouton, Berlin, Boston, 1990. URL: https://doi.org/10.1515/9783110870374-007, doi:doi:10.1515/9783110870374-007.

[27] J. R. Hurford. The evolution of the critical period for language acquisition. *Cognition*, 40(3):159–201, 1991.

[28] G. Kachergis, G. Loukatou, and M. C. Frank. Identifying the distributional sources of children's early vocabulary. In J. Culbertson, A. Perfors, H. Rabagliati, and V. Ramenzoni, editors, *Proceedings of the 44th Annual Meeting of the Cognitive Science Society*, pages 3202–3208. Cognitive Science Society, 2022.

[29] D. P. Kingma and M. Welling. Auto-encoding variational bayes, 2022. URL: https://arxiv.org/abs/1312.6114, arXiv:1312.6114.

[30] S. Kirby. Spontaneous evolution of linguistic structure-an iterated learning model of the emergence of regularity and irregularity. *IEEE Transactions on Evolutionary Computation*, 5(2):102–110, 2001. doi:10.1109/4235.918430.

[31] S. Kirby, H. Cornish, and K. Smith. Cumulative cultural evolution in the laboratory: An experimental approach to the origins of structure in human language. *Proceedings of the National Academy of Sciences*, 105(31):10681–10686, 2008.

[32] S. Kirby and J. R. Hurford. The emergence of linguistic structure: An overview of the iterated learning model. *Simulating the Evolution of Language*, page 121–147, Mar 2002. doi:10.1007/978-1-4471-0663-0_6.

[33] P. K. Kuhl. Is speech learning 'gated' by the social brain? *Developmental science*, 10(1):110–120, 2007.

[34] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman. Building machines that learn and think like people, 2016. URL: https://arxiv.org/abs/1604.00289, arXiv:1604.00289.

[35] E. H. Lenneberg. The biological foundations of language. *Hospital Practice*, 2(12):59–67, 1967.

[36] E. L. Newport. Maturational constraints on language learning. *Cognitive Science*, 14(1):11–28, 1990.

[37] M. Oliphant. The learning barrier: Moving from innate to learned systems of communication. *Adaptive Behavior*, 7(3-4):371–383, 1999.

[38] M. Oliphant and J. Batali. Learning and the emergence of coordinated communication. *Center for Research on Language Newsletter*, 11, 03 1997.

[39] S. Pinker. *The Language Instinct: How the Mind Creates Language*. Penguin UK, 2003.

[40] E. Rosch. Cognitive representations of semantic categories. *Journal of Experimental Psychology: General*, 104(3):192, 1975.

[41] G. Sains, C. Houghton, and S. Bullock. Spatial community structure impedes language amalgamation in a population-based iterated learning model, 2023. arXiv:2305.11962.

[42] K. Smith, S. Kirby, and H. Brighton. Iterated learning: A framework for the emergence of language. *Artificial Life*, 9(4):371–386, 2003.

[43] C. Snow, A. Arlman-Rupp, Y. Hassing, J. Jobse, J. Joosten, and J. Vorster. Mothers' speech in three social classes. *Journal of Psycholinguistic Research*, 5:1–20, 01 1976. doi:10.1007/BF01067944.

[44] M. Tomasello. *Constructing a language: A usage-based theory of language acquisition*. Harvard university press, 2005.

[45] J. Vanhove. The critical period hypothesis in second language acquisition: A statistical critique and a reanalysis. *PLOS ONE*, 8(7):1–15, 07 2013. doi:10.1371/journal.pone.0069172.

[46] F. Xu, S. Carey, and J. Welch. Infants' ability to use object kind information for object individuation. *Cognition*, 70(2):137–166, 1999. doi:10.1016/S0010-0277(99)00007-4.

# Appendix A

# Appendix A: AI Prompts/Tools

- I used ChatGPT to develop the initial structure and code for TikZ diagrams presented in this dissertation, which I subsequently refined and customised.
- I used Claude and Grammarly to identify my grammatical errors and unclear phrases.
- I used ChatGPT to debug my code implementations.
- I used Claude to understand some of the articles.